

SMART INDIA HACKATHON 2026

Problem Statement 26123

Team Preparation & Presentation

Readiness Document

Edge-AI Based Distributed Fleet Coordination for Autonomous
Mobile Robots (AMRs) in Smart Warehouses

September 08, 2026

Table of Contents

Click any entry to jump to that chapter. See "How to Use This Document" for the recommended reading order and which role chapter is compulsory for whom.

Part I — Common Chapters (compulsory for every team member)

How to Use This Document

Project & Problem Statement

End-to-End Architecture Walkthrough

Architecture Decisions & Alternatives

The Benchmark Journey & Methodology

Judge Q&A Preparation

Glossary

Part II — Role-Specific Chapters

Path Planning — compulsory for Member 1

Robot Communication — compulsory for Member 2

Coordination — compulsory for Member 3

Frontend — compulsory for Member 4

Edge / ECE — compulsory for Member 5

Backend — compulsory for Member 6

PART I

Common Chapters

Compulsory reading for every team member — the problem statement, the real end-to-end system trace, architecture decisions, the full benchmark journey, judge Q&A preparation, and a glossary.

How to Use This Document

What this document is

This is the single preparation document for presenting **SIH 2026, Problem Statement 26123 — Edge-AI Based Distributed Fleet Coordination for Autonomous Mobile Robots (AMRs) in Smart Warehouses** to a judging panel. It exists so that every team member can explain the whole system — not just their own piece of it — cold, without notes, while still being able to go into real depth on their own role if asked.

It is split into two parts.

Part I — Common Chapters. Compulsory reading for **every** team member, regardless of role. These chapters cover the problem statement itself, a concrete end-to-end trace of how a request actually flows through the real system, the major architecture decisions and why the alternatives were rejected, the full and sometimes messy story of how the benchmark numbers were produced (including the mistakes made and caught along the way), a prepared set of judge questions and answers, and a plain-language glossary.

Part II — Role-Specific Chapters. One chapter per module: Path Planning, Robot Communication, Coordination, Frontend, Edge/ECE, Backend. Each chapter is compulsory for the member who owns that module, and optional-but-encouraged reading for everyone else. Each opens with a one-line banner stating exactly who it's compulsory for.

Recommended reading order

1. Read all of Part I first, in order. The **Benchmark Journey & Methodology** chapter is the single most important chapter in the document — if a judge asks only one hard question, it is very likely to land somewhere in that chapter's territory.
2. Then read your own Part II chapter in full.
3. Skim the other five Part II chapters — you don't need their implementation depth, but you should be able to answer "what does that module do and how does it talk to mine" for every other module, and Part I's architecture walkthrough already gives you most of that.
4. Re-read the **Judge Q&A** chapter the night before the demo. It is written so the short "verified answer" is judge-ready on its own, with a natural follow-up already anticipated.

Which role chapter is compulsory for whom

Team member	Compulsory Part II chapter
Member 1	Path Planning
Member 2	Robot Communication
Member 3	Coordination
Member 4	Frontend
Member 5	Edge / ECE

Team member	Compulsory Part II chapter
Member 6	Backend

A note on honesty in this document

Every number, file path, function name, and event name in this document was verified directly against the actual source code, the actual benchmark result files (`summary.json` / `raw_results.csv` in the `benchmark_results` family of directories), or `CLAUDE.md` 's own recorded audit trail — not reconstructed from memory or assumption. Where a fact could not be independently verified from a surviving primary source (there is exactly one such case — the very first, pre-fix benchmark run, whose raw output file was overwritten the same day it was produced), this document says so explicitly in the Benchmark Journey chapter rather than presenting a reconstructed number with false confidence. If you ever find a number in this document that disagrees with what you see in the running system, trust the running system and flag the document — it is meant to describe reality, not the other way round.

Project & Problem Statement

The literal problem statement

SIH 2026, Problem Statement 26123: "Edge-AI Based Distributed Fleet Coordination for Autonomous Mobile Robots (AMRs) in Smart Warehouses." Sponsor: **Bharat Electronics Limited**. Category: **Software**. Theme: **Smart Automation**.

This exact title and sponsor/category/theme triple appears identically at the top of `README.md` and `SIH_26123_Project_Overview.md` — both are consistent, and we treat this as the verbatim reference text, since no separate raw PS document exists in the repository to quote from instead.

What it is actually asking for

Read plainly, the title asks for four things at once, and the project's own documentation (`SIH_26123_Project_Overview.md` §1, §19, §20) makes each one concrete:

1. **"Distributed" / "Edge-AI"** — robots must make local, real-time movement and conflict-resolution decisions and communicate directly with nearby robots, rather than depending on a central server for every step. "Edge-AI" here does not require an on-device neural network; the project's documented position (§10 of the overview) is that decentralized, local decision-making on each robot already qualifies as edge intelligence, since the category is Software and no physical hardware is required.
2. **"Fleet Coordination"** — the system must handle the actual hard problems of multiple robots sharing a space: conflict/deadlock avoidance at choke points, dynamic re-routing around a newly blocked aisle, and task allocation/re-assignment when a robot becomes unavailable.
3. **A measurable improvement claim** — the project's self-imposed, predeclared success bar (`SIH_26123_Project_Overview.md` §19, §21) is **zero referee-verified inter-robot collisions**, and **≥20% completion-time (makespan) reduction versus a `STOP_AND_WAIT` baseline specifically on overlapping-path scenarios** — not a vague "our system is better" claim.
4. **A live, working demonstration** — a fleet dashboard showing the system operating, and specifically a demonstration that the decentralized coordination layer survives the central backend being killed mid-run.

What "at least 3 AMRs" and "dynamic warehouse" mean in this implementation

The project's scope (`SIH_26123_Project_Overview.md` §1) targets **4–6 AMRs for demo purposes** — comfortably above the "at least 3" floor. The system currently exercises up to **6 concurrent robots** in `g_high_load` and `i_parallel_aisles`, and 1–3 robots in the more targeted conflict-isolation scenarios (`b_intersection`, `c_narrow_aisle`, `d_deadlock`, `e_blocked_aisle`, `f_task_reassignment`) — the smaller robot counts there are a deliberate test-isolation choice, not a

scale limitation; `h_negative_control` uses 3 robots specifically to also serve the "at least 3" requirement in a no-conflict control case.

"Dynamic warehouse" is implemented as a **20×15 grid map** (`simulation/warehouse/demo_map.py`) with deliberately placed choke points (two single-file vertical aisles at columns 2 and 12, and a horizontal cross-aisle at row 7) that force real contention — not a sparse map where robots rarely meet, which the project's own role documentation (`role/01_PATH_PLANNING.md` §8.1) explicitly flags as a way to make the whole improvement claim look weak by making stop-and-wait perform almost as well as anything else. "Dynamic" specifically means: (a) an aisle can become blocked mid-run via a scripted `AISLE_BLOCKED` event (scenario `e_blocked_aisle`), forcing a real re-route; and (b) a robot can go fully offline mid-task via simulated sensor/heartbeat failure (scenario `f_task_reassignment`), forcing a real task hand-off to another robot.

What the system demonstrably does today (and what it doesn't claim)

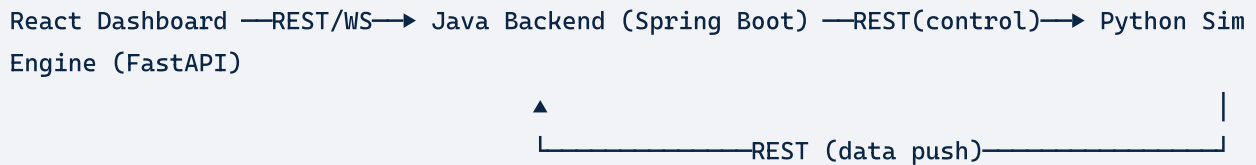
Verified directly against the running code and the audited benchmark (see the Benchmark Journey chapter for full detail):

- **Zero referee-verified collisions:** confirmed across the full, current 270-run benchmark (9 scenarios × 3 modes × 10 seeds) — independently re-summed from the raw CSV, not just the generated report.
- **≥20% makespan reduction:** met on one specific scenario, `i_parallel_aisles` (23 → 18 ticks, 21.74%), reproducible across all 10 audited seeds. **Not** met as a pooled, blended, or "overall system" number — the project is explicit about this distinction, and so is this document (see the Benchmark Journey chapter).
- **Real decentralized coordination surviving backend loss:** every collision/deadlock/makespan number in the benchmark was produced with the Java backend's URL pointed at an intentionally unreachable port — movement coordination never depends on it being up.
- **Real dynamic re-routing and task reassignment:** `e_blocked_aisle` and `f_task_reassignment` both complete only under the proposed decentralized system; both baselines genuinely time out facing the identical scripted event (this is itself a deliberately corrected, not favorable-by-default, measurement — see the Benchmark Journey chapter for why that mattered).
- **Frontend:** a real, working React dashboard that talks only to the Java backend (never directly to the Python simulation engine), showing live warehouse state, robot status, real per-robot negotiation decisions, and a separate, clearly-labeled offline benchmark proof view.

What is **not** claimed: an on-device neural network anywhere in the pipeline; a fully decentralized task-assignment layer (task assignment is deliberately centralized — see the Architecture Decisions chapter for why); a blended "our system is X% faster overall" number; or physical hardware of any kind (explicitly out of scope, per §2 of the project overview — the category is Software).

End-to-End Architecture Walkthrough

Every file, class, function, and endpoint name below was read directly from the running source during this document's preparation (not inferred or approximated). The three-process shape is fixed and frozen:



The dashboard **never** talks to Python directly — every dashboard request goes through the Java backend, and this is enforced by construction (`dashboard/src/api/client.ts` hardcodes `BASE_URL = 'http://localhost:8080'`, the Java backend's port; there is no Python URL anywhere in the frontend).

Flow A — Simulation start-to-first-tick

1. A user picks a scenario, mode, seed, and speed in the dashboard's left panel (`dashboard/src/components/scenario/ScenarioDrawer.tsx`, `ModeSelector.tsx`, `SeedSpeedControls.tsx`) and clicks **START** (`dashboard/src/components/shell/PlaybackControls.tsx`).
2. `useSimulationControl.ts`'s `start()` calls `startSimulation()` in `dashboard/src/api/client.ts`, which issues `POST http://localhost:8080/api/simulation/control` with body `{action: "start", scenarioId, mode, seed, speed}`.
3. Java's `SimulationControlController.control()` receives this. If `action == "start"`, it calls `TaskAllocationService.currentAssignmentsForSimulation()` to fetch any real, currently ASSIGNED/IN_PROGRESS `Task / TaskAssignment` rows from its own database, then calls `SimulationClient.start(scenarioId, mode, seed, speed, assignments)`.
4. `SimulationClient` (a thin Spring `RestClient` wrapper, default target `http://localhost:8001`, configurable via `simulation.engine.url`) issues `POST /control/start` to the Python FastAPI server, with body `{scenarioId, mode, seed, speed, taskAssignments}`.
5. Python's `simulation/runner.py` exposes this route directly (`@app.post("/control/start")`). It rejects the call with `400 {"error": "Simulation is already running."}` if one is already in progress (this is a real, observed behavior — Java surfaces it back to the dashboard as a clear `SIMULATION_ENGINE_ERROR`, not a crash). Otherwise it spawns a background thread running `_runner.run(scenario_id, mode, seed, speed, task_assignments)` and returns immediately with an ack.
6. Inside `SimulationRunner.run()`: it loads the named scenario via `simulation.scenarios.get_scenario()`, applies any passed `task_assignments` on top of the scenario's own hardcoded robot start/goal pairs (`_apply_task_assignment()`), constructs one `GridStarPlanner`, one `InProcessBus` (wrapped in `_MessageCountingTransport`), one

`SimulationSensorSource` per robot, and one `RobotAgent` per robot (composing `Planner`, `Transport`, `SensorSource`, `ConflictDetector`, `ConflictResolver`, `DeadlockDetector` — this exact composition list is `RobotAgent.__init__` 's real constructor signature in `robot_agent/agent.py`). It also pushes the scenario's real `WarehouseMap` to Java via `POST /api/warehouse`.

7. Java's `WarehouseController.setWarehouseMap()` receives this and caches it (`WarehouseCacheService`). The dashboard's `useWarehouseMap.ts` hook (`GET /api/warehouse`) had been getting `503 WAREHOUSE_MAP_UNAVAILABLE` until this point — it refetches on scenario change and now gets the real map.
8. The tick loop begins: `for tick in range(1, max_ticks+1)`. Each robot's `RobotAgent.tick(tick)` runs (see Flow B for what happens inside). At the end of every tick, `SimulationRunner` batches all robots' `RobotState` and calls `_push_to_backend("/api/telemetry/batch", ...)`.
9. Java's `TelemetryIngestController.ingest()` receives the batch, updates `RobotCacheService` (the in-memory "latest state" store), persists a `RobotEntity` row per robot, samples a `TelemetryEntity` row every 10th tick, and — critically — calls `LiveUpdateBroadcaster.broadcastRobotStateBatch(tick, batch)` in the **same request**, which serializes `{"type": "ROBOT_STATE_BATCH", "tick": ..., "payload": [...]}` and pushes it to every open `/ws/live` WebSocket session via `LiveUpdateWebSocketHandler.broadcast()`.
10. The dashboard's `useLiveFeed.ts` hook receives this frame over its already-open WebSocket, appends it to its tick buffer keyed by `msg.tick`, and `WarehouseCanvas.tsx` re-renders the SVG map with the new robot positions. This is genuinely live, tick-by-tick — no polling anywhere in this path.

Flow B — A conflict being detected and resolved (inside one `RobotAgent.tick()` call, `DECENTRALIZED_PROPOSED` mode only)

This is the actual sequence inside `robot_agent/agent.py::RobotAgent.tick()`, traced line-by-line:

1. `telemetry = self.sensor.read(self.robot_id, current_tick)` — `edge/sensor_source.py` 's `SimulationSensorSource.read()` derives a (possibly noisy/dropped/offline) reading from ground truth.
2. `self.state = self._apply_telemetry(...)` folds it into the robot's `RobotState`.
3. If a replan is needed, `self.planner.plan(...)` — `simulation/runner.py` 's own `GridAStarPlanner.plan()`, a 4-connected grid A* supporting the same `avoid_intervals` space-time mechanism — computes a fresh `RobotPath`. **Precise, verified detail:** `GridAStarPlanner` is a separate class `simulation/runner.py` implements itself (explicitly commented in its own source as "conforming to Member 1's Planner interface") — it is not an import of `planner/astar.py::Planner`. That class is real, tested Member 1 code (with a fuller feature set — `.replan()`, `PlanningFailedError`, `InvalidMapError`, `validate_path()`), but it is not wired into the live/benchmark execution path at all — confirmed by grep, `simulation/runner.py` contains zero imports from `planner.astar` and `robot_agent/agent.py` never calls `.replan()` on anything. `planner/centralized_baseline.py::plan_centralized()`, by contrast, genuinely is imported

and used directly, for the `CENTRALIZED_RESERVATION` mode. See the Path Planning role chapter for the full detail on this distinction.

4. `incoming = self.transport.receive(self.robot_id)` — pulls this tick's already-delivered peer intent messages from the `InProcessBus` (`robot_agent/communication/transport.py`).
5. `self._peer_held_cells = self._collect_peer_held_cells(incoming)` rebuilds, from peers' own broadcast position/status, which cells are physically occupied by a stationary peer right now.
6. `conflict = self.detector.detect(self.state, self._current_path_obj(), incoming)` — `collision_engine/detection.py`'s `ConflictDetector.detect()` compares this robot's own planned waypoints against every peer's broadcast `plannedPath`, checking (in order of severity/tick) `SAME_CELL`, `CROSSING` (edge-swap), `NARROW AISLE HEADON`, and a fourth, purpose-built check — stationary-peer occupancy — for a peer that has stopped and so no longer has the contested cell anywhere in its own broadcast path.
7. If a `Conflict` was found, `action = self.resolver.resolve(conflict, self.state, incoming)` — `collision_engine/resolution.py`'s `ConflictResolver.resolve()` applies five deterministic priority rules in strict order (emergency/priority-5 task → higher `Task.priority` → already inside the contested cell → lower rerouting cost → `robotId` lexicographic tie-break) and returns one of `CONTINUE` | `WAIT` | `YIELD` | `REROUTE` | `REASSIGN_TASK`. `_apply_action()` then updates `self.state.status` and internal bookkeeping (`_waiting_on`, `_temporary_avoid_cells`) accordingly.
8. `self.deadlock.check(self.robot_id, self._stall_ticks, waiting_on)` — `collision_engine/deadlock.py`'s `DeadlockDetector`, a simple N-tick (default 5) stall heuristic — may force `_force_yield()`, which reroutes only the lower-priority robot in the pair (via `_should_step_aside()`, mirroring the resolver's own priority order) so both sides don't dodge into each other.
9. `self._advance_one_cell_if_clear(conflict)` performs the actual movement, re-checking physical peer occupancy and blocked cells unconditionally regardless of what the resolver decided — this is what stops a `CONTINUE` verdict from ever driving through a peer that is still physically parked in the target cell.
10. `self.transport.broadcast(build_intent_message(self.state))` — `robot_agent/communication/messages.py`'s `build_intent_message()` — publishes this robot's own new state (position, destination, `plannedPath`, priority, status) onto the bus for every other robot's next tick.
11. Back in `simulation/runner.py`, if this robot's `_action_this_tick` genuinely changed from its last-recorded value, `_emit_negotiation_event_if_changed()` appends a `NEGOTIATION SimulationEvent` (`{robotId, peerRobotId, resolutionAction}`) — purely observational, reading attributes the agent already computed, never a new decision.
12. Independently of all of the above (and never influenced by it), `simulation/runner.py`'s tick loop also calls `self.referee.check(gt_positions)` — `simulation/referee.py`'s `CollisionReferee`, which imports nothing from `collision_engine` and compares raw ground-truth positions directly — to certify whether an actual collision happened this tick. This is the number that becomes `PerformanceMetric.collisionCount`, and it is structurally incapable of being fooled by a bug in `ConflictDetector`, because it never calls into it.

This whole chain (steps 4–10) is what "decentralized" means concretely in this system: every decision is a function of `own_state` plus messages received over the bus, never a shared global robot list — verified structurally, not just claimed, by

`ConflictDetector.detect()` / `ConflictResolver.resolve()` / `DeadlockDetector.check()` 's actual signatures, none of which accept anything like an `all_robots` parameter.

Flow C — A scenario completing and metrics reaching the dashboard

1. `SimulationRunner.run()` 's tick loop ends when either `all_done` (every robot `IDLE` /at destination, or `OFFLINE` with its task genuinely reassigned) becomes true, or `max_ticks` is reached (a timeout — recorded as `completed=False`, never given a fabricated makespan).
2. The runner assembles a `PerformanceMetric` (`runId`, `scenarioId`, `mode`, `totalCompletionTicks`, `avgCompletionTicks`, `collisionCount`, `deadlockCount`, `rerouteCount`, `idleTicksTotal`, `messageCount`, `seed`) and calls `_push_to_backend("/api/metrics", ...)`.
3. Java's `MetricsController.ingestMetric()` persists a `PerformanceMetricEntity` row and calls `LiveUpdateBroadcaster.broadcastMetric(tick, metric)`, sending a `{"type": "METRIC_UPDATE", ...}` frame over `/ws/live`.
4. The dashboard's live view doesn't currently render `METRIC_UPDATE` into the live panel (it's reserved for the separate Benchmark & Proof view, which instead shows the offline, already-audited 270-run result — see the Benchmark Journey chapter for exactly why those two are kept deliberately separate). The frame still arrives and is captured by `useLiveFeed.ts`, confirming the full pipe works end-to-end even though this particular payload isn't yet surfaced live.
5. Separately, at any time, `GET http://localhost:8080/api/metrics?scenarioId=...` (`dashboard/src/api/client.ts::fetchMetrics()`) returns every `PerformanceMetric` Java has ever ingested for that scenario across modes — this is what a live re-run's own numbers would be inspected through, distinct from the static, pre-computed benchmark artifact the Benchmark & Proof view actually displays.

A structural point worth stating explicitly

Nowhere in Flow B does any code accept a full robot roster and decide on their collective behalf. The one place central coordination genuinely exists is Flow A step 3 — Java's `TaskAllocationService`, which decides *which task goes to which robot*, not *how two robots avoid colliding*. This is the literal, code-verified boundary of the "hybrid centralized/decentralized" claim — see the Architecture Decisions chapter for the reasoning behind drawing the line exactly there.

Architecture Decisions & Alternatives

Each decision below is written so a judge's "why didn't you just do X instead" has a ready, specific answer — not a vague appeal to "best practice."

1. Hybrid centralized task assignment / decentralized movement coordination

Chosen: task assignment (which robot does which pickup/delivery) lives in the Java backend, centralized. Movement-level conflict detection, resolution, and deadlock recovery live in each robot's own agent process, decentralized — no per-tick central arbiter of any kind.

Alternatives considered: - *Fully centralized* (a single scheduler also decides every robot's move, every tick): rejected because it directly contradicts the problem statement's own framing — "robots must make local, real-time movement and conflict-resolution decisions" — and because it reintroduces the single-point-of-failure the project is explicitly trying to avoid for the safety-critical part of the system (movement). It would also make "kill the backend mid-demo and watch robots keep avoiding each other" impossible to demonstrate, which is the project's own headline resilience proof ([SIH_26123_Project_Overview.md](#) §3). - *Fully decentralized, including task assignment* (robots negotiate amongst themselves who does which task): rejected deliberately, and the project's own overview is explicit about why (§3): task assignment needs *global* visibility to optimize well, and a bad assignment simply gets reassigned later at no safety cost — there is no resilience argument for decentralizing it, only added complexity. The project's stated principle is "be honest about what's centralized and what's decentralized, instead of pretending everything is decentralized" — real commercial AMR fleets (the overview names Amazon Robotics, Locus, Geek+, OTTO) run centralized fleet managers as standard practice, because LAN latency (1–5ms) is irrelevant at warehouse robot speeds. The actual argument for decentralizing *movement* specifically is resilience, not latency, and that argument doesn't apply to task assignment.

Where the line is actually drawn, precisely: verified in code (see the Architecture Walkthrough chapter, Flow A step 3 vs. Flow B) — Java's `TaskAllocationService` decides robot↔task pairing; every conflict/deadlock decision inside a tick is a function of that one robot's own state plus messages received over the bus, never a shared robot list.

2. Python / Java / TypeScript three-language split

Chosen: Python for the simulation engine (planner, robot agents, coordination, edge), Java + Spring Boot for the backend, TypeScript + React for the dashboard.

Alternative considered: a single-language stack (e.g., everything in Python, or everything in Java/Kotlin, or a Node.js backend to match the TypeScript frontend). Rejected for three concrete reasons, not just "everyone already knew a different language": (a) the team's actual skill distribution — six members each with an assigned specialty; (b) Java/Spring Boot's REST + WebSocket + JPA/persistence tooling is genuinely mature for exactly the backend's job (task management, ingestion, a live-update facade) with very little boilerplate; (c) Python's ecosystem is the natural fit for the

simulation/planning/coordination logic (data structures, A* search, dataclasses-as-contracts) and for rapid iteration on the actual research-adjacent part of the project — the benchmark methodology itself went through several corrections (see the Benchmark Journey chapter), which would have been much slower to iterate on in a more verbose language. The cost of a three-language split — needing mirrored DTOs/interfaces in each language — is real and explicitly acknowledged (`docs/00_SHARED_CONTRACTS.md`), and paid for deliberately: `shared/python/models.py` is the single canonical source, and the Java/TypeScript copies are documented as maintained manually, with "if this file and the doc ever disagree, the code wins" as the tiebreak rule.

3. In-process publish/subscribe bus as the default transport (not real network sockets)

Chosen: `robot_agent/communication/transport.py`'s `InProcessBus` — a per-robot in-memory queue, sitting behind an abstract `Transport` interface (`send` / `broadcast` / `receive`).

Alternative considered: real UDP or WebSocket sockets between separate robot processes. Rejected for the *default* path for a concrete, named risk: a live SIH demo on unfamiliar venue Wi-Fi is a real failure mode, and a real network transport is exactly the kind of thing that can fail unpredictably in front of judges for reasons that have nothing to do with the actual coordination logic being demonstrated. `InProcessBus` is deterministic and immune to that risk while still enforcing the actual decentralization property that matters for the demo: agents only ever see each other through explicit messages (`Transport.receive()`), never shared memory or a global robot registry — verified structurally in `RobotAgent.tick()`'s and `ConflictDetector.detect()`'s signatures. The `Transport` abstract base class exists specifically so a real `UDPTTransport` / `WebSocketTransport` *could* be substituted later behind the identical interface for an optional hardware bonus segment, without touching any caller — this was designed for, not just left open by accident, but was never required or built, since hardware is explicitly optional and isolated.

4. Discrete-tick simulation model (not continuous/real-time)

Chosen: the simulation advances in discrete integer ticks; at most one cell of movement per robot per tick. Two speeds: `LIVE` (throttled, ~2 ticks/sec, `time.sleep(0.5)` between ticks in `simulation/runner.py`, for the judge-facing demo) and `BATCH` (unthrottled, for the repeated evaluation runs — this is what every benchmark number in this document was produced with).

Alternative considered: continuous-time simulation with real robot kinematics/physics. Rejected because it adds substantial complexity (integration, collision geometry beyond grid-cell overlap, timestep-size sensitivity) that doesn't serve the actual claims being made — the project's success criteria (zero collisions, $\geq 20\%$ makespan reduction) are both naturally and unambiguously expressible in tick counts, and discrete ticks make every benchmark run exactly, bit-for-bit reproducible given a seed — which turned out to matter a great deal, since reproducibility across all 10 seeds is one of the concrete pieces of evidence backing the final 21.74% result (see the Benchmark Journey chapter). A continuous model would have made "is this the same result or did floating-point/timing noise change it slightly" a much harder question to answer cleanly.

5. Grid-based map representation (not graph-based)

Chosen: the warehouse is a 2D grid (`WarehouseMap.gridWidth × gridHeight` , each cell walkable or an obstacle), not an arbitrary graph of named nodes/edges.

Alternative considered: a graph representation (nodes = named locations, edges = traversable connections with costs). Rejected per the project's own stated reasoning (`role/01_PATH_PLANNING.md` §8.1): a grid is simpler to reason about, trivial to render directly on the dashboard (the actual `WarehouseCanvas.tsx` SVG renderer works purely from `gridWidth / gridHeight` plus cell lists — no separate layout algorithm needed), and standard for warehouse-aisle-style layouts specifically. A graph would have added a translation layer between "what the planner reasons about" and "what the dashboard draws" for no benefit this project actually needed — grid cells map 1:1 onto both A* states and SVG rectangles.

6. Three-mode comparison design (`STOP_AND_WAIT` , `CENTRALIZED_RESERVATION` , `DECENTRALIZED_PROPOSED`)

Chosen: every scenario is run under all three modes, same map/task batch/seed (paired comparison), so the proposed system is compared against *two* baselines, not one or none.

Alternative considered: compare only against `STOP_AND_WAIT` (a naive reactive-occupancy baseline), or against nothing at all (just demonstrate the proposed system). Rejected because a single weak baseline invites exactly the judge question this project pre-empts by design: "is `STOP_AND_WAIT` artificially weak so you look good?" (see Judge Q&A #11). `CENTRALIZED_RESERVATION` (`planner/centralized_baseline.py::plan_centralized()` — sequential prioritized A* against one shared space-time reservation table, provably free of both vertex *and* edge/swap conflicts by construction) is a strong, realistic baseline representative of how a real centralized AMR fleet manager would actually plan, not a strawman. Comparing against both simultaneously is what lets the project honestly report, for example, that `STOP_AND_WAIT` structurally fails to complete several contention scenarios at all (a real completion-rate finding) while `CENTRALIZED_RESERVATION` does complete them but the decentralized system still holds its own — a richer, harder-to-dismiss picture than a single-baseline comparison could give.

7. SQLite for local dev, Postgres-compatible via JPA (not a hardcoded SQLite-only design)

Chosen: `backend/src/main/resources/application.properties` configures SQLite (`jdbc:sqlite:./sih26123.db`) via Spring Data JPA + Hibernate, with the properties file's own comment stating the Postgres swap is "three lines + remove sqlite-jdbc dependency."

Alternative considered: build directly against Postgres from the start, or hand-roll a custom persistence layer. Rejected because SQLite has zero setup friction for a six-person team all running the stack locally on their own machines during development (`role/06_BACKEND.md` §8.6) — no separate database server to install or configure — while JPA/Hibernate as the abstraction layer means the swap to a "real" production database is a configuration change, not a rewrite, if that were ever needed. No

SQLite-specific SQL is hand-written anywhere in the codebase (verified: the entity/repository layer is pure Spring Data JPA), which is exactly what keeps that swap genuinely cheap rather than a stated intention that would break in practice.

The Benchmark Journey & Methodology

This is the most important chapter in this document. Every number below was independently re-derived from the surviving raw benchmark artifacts (each `benchmark_results` directory's own `raw_results.csv` and `summary.json`) during this document's preparation, not copied from a prior report without re-checking — with exactly one disclosed exception, flagged where it occurs.

What a tick is, and its exact role in every calculation

A **tick** is one discrete simulation step. In one tick, each robot moves at most one grid cell (or waits in place). Every timing number in this entire document — every "makespan," every "23 ticks," every deadline — is a count of ticks, not seconds or any real-world time unit. Two run speeds exist (`simulation/runner.py`): `LIVE` sleeps ~0.5s between ticks purely so a human watching a demo can see movement; `BATCH` has no artificial delay and is what every benchmark number in this chapter was produced under. **Makespan** is the number of ticks from the start of a run until the *last* robot reaches its destination — the standard multi-agent pathfinding (MAPF) metric, and the one the SIH statement's "≥20% completion-time reduction" criterion is measured against. A run that never gets every robot home within the scenario's own predeclared `max_ticks` is a **timeout** — it has no makespan at all; it is recorded as `completed=False`, never silently given a fabricated tick count equal to the deadline. This distinction — a genuinely completed run's real tick count vs. a timed-out run's explicit non-number — is the single fact that the entire "why did your number change from +31.55% to a negative number" story (below) turns on.

What each of the 9 scenarios tests

Read directly from the scenario files in `simulation/scenarios/`:

1. **a_normal** — two robots on independent, non-overlapping rows. No scripted conflict. A negative control: proves normal movement itself works and produces no false positives.
2. **b_intersection** — two robots' straight-line paths cross at exactly one shared cell, at different priorities (3 vs 1). Tests basic conflict detection and deterministic priority resolution at a single, unavoidable bottleneck.
3. **c_narrow_aisle** — two robots enter a single-file aisle from opposite ends, walking straight at each other. Tests the `NARROW AISLE HEADON` conflict type and forced yield.
4. **d_deadlock** — a short head-on corridor with an added one-tick message delay (`MessageFaultConfig(delay_ticks=1)`), specifically engineered so both robots can genuinely stall waiting on each other. Tests the `DeadlockDetector`'s N-tick stall heuristic and forced-yield recovery.
5. **e_blocked_aisle** — one robot's route, with a scripted `AISLE_BLOCKED` event injected mid-run (tick 4) directly in its path. Tests dynamic replanning around a newly-appeared obstacle.
6. **f_task_reassignment** — robot R1 is scripted to go fully `OFFLINE` at tick 5 mid-task, with an idle backup robot R2 standing by. Tests mid-run task hand-off.

7. **g_high_load** — six robots in three simultaneous head-to-head pairs — the fleet-scale stress test.
8. **h_negative_control** — three robots on three widely-separated, non-overlapping rows. A second negative control, specifically to pre-empt "did you only test easy/favorable cases?"
9. **i_parallel_aisles** — six robots, three parallel single-file aisles of equal entrance-distance, one geometrically-shortest "direct" lane and two tied "detour" lanes, unidirectional (so **STOP_AND_WAIT** queues rather than timing out). This is the scenario the final $\geq 20\%$ headline result comes from — see below for exactly why it, and not **b_intersection**, is the one where the target is met.

The five categories, and precisely what each one claims

The benchmark computes five deliberately separate measures. No team member should ever quote one category's number as if it were another's:

Category	Scenarios	What it actually claims
1 — Overlapping-path throughput	b_intersection , i_parallel_aisles	A scoped percentage : makespan reduction vs. STOP_AND_WAIT , computed <i>only</i> over runs where both modes complete. This is the only category that feeds the $\geq 20\%$ headline.
2 — Completion under contention	c_narrow_aisle , d_deadlock , g_high_load	A completion rate , not a speed number. STOP_AND_WAIT completes 0% of these; the proposed system and the centralized baseline both complete 100%.
3 — Resilience & re-routing	e_blocked_aisle , f_task_reassignment	A capability : does the system detect and recover from a scripted dynamic event at all? Both baselines correctly time out facing the identical event (they have no re-routing/reassignment mechanism); only the proposed system completes.
4 — Collision safety	all 9 scenarios	An absolute safety count — referee-verified collisions, full stop. Not a percentage of anything.
5 — Negative controls	a_normal , h_negative_control	A parity check : proof of zero regression where zero conflict is even possible, never cited as throughput evidence for anything.

The journey, stage by stage, with exact numbers

Stage 0 — the original run (2026-09-04, real collisions found)

The very first full 240-run benchmark found **real, referee-verified collisions**:

DECENTRALIZED_PROPOSED was collision-free in only 60/80 runs (**d_deadlock** and **g_high_load** collided in all 10/10 seeds each, fully reproducible — not intermittent), and even

CENTRALIZED_RESERVATION collided in 3 scenarios, directly contradicting its own docstring's guarantee at the time. **Disclosure**: this run's raw output file was overwritten the same day by the next run (both

used the same default `benchmark_results/` directory), so its exact numbers are traceable only to the project's own contemporaneous written record, not to a surviving CSV I could re-derive myself. Every later stage below, I did independently re-verify from a surviving raw file.

Two root causes were found and fixed: - **DECENTRALIZED_PROPOSED 's stationary-peer blind spot**: a robot that stops popped its current cell off its own broadcast path, so `ConflictDetector` had no way to see "a peer is sitting right here" — another robot could be told `CONTINUE` and drive straight into it. Fixed with `ConflictDetector._check_stationary_occupancy()` plus a physical movement guard in `RobotAgent._advance_one_cell_if_clear()` that blocks entry into a peer-held cell regardless of what the resolver decided. - **CENTRALIZED_RESERVATION 's reservation table only tracked vertex conflicts** (same cell, same tick), not edge/swap conflicts (two robots crossing the same edge in opposite directions between consecutive ticks) — a distinct, classic MAPF conflict type. Fixed in `planner/centralized_baseline.py::_astar_reserved()`, which now also rejects a move that would swap places with a robot coming the other way.

Stage 1 — post-fix, but a flawed percentage (`benchmark_results/` , 2026-09-04 23:13, 993.5s)

With both collision bugs fixed, the same 240-run benchmark now showed **0/240 runs colliding across all three modes** — a genuine, durable result that held for every subsequent stage. But the reported headline was `sihVerdict: "VERIFIED – pooled makespan improvement 31.55% >= 20% target"`. This number was wrong, and the reason is precise: the methodology at this point counted a *timed-out* run as if it had *completed exactly at the deadline* (`maxTicks`) — so `STOP_AND_WAIT` timing out at 50 ticks on `c_narrow_aisle` was averaged into the pool as "STOP_AND_WAIT took 50 ticks," not excluded as "STOP_AND_WAIT never finished." Since `STOP_AND_WAIT` genuinely fails to complete several contention scenarios, this inflated its apparent makespan enormously, which inflated the *proposed* system's apparent improvement over it — the opposite of a fair reading. An independent audit at this stage correctly flagged the benchmark as "**valid but the claim needs different presentation**," not invalid outright.

Stage 2 — corrected methodology, first honest negative result (`benchmark_results_final/` , 2026-09-05 23:13, 986.5s)

Fixed: (a) completion/timeout made explicit — a `RunResult.makespanTicks` is `None`, never a substituted number, whenever `completed=False`; (b) `PerformanceMetric.deadlockCount` reconnected to the *real* `DeadlockDetector.check()` activations (it had been silently reading a different, disconnected proxy condition that could read 0 while the real detector had genuinely fired twice); (c) the run's actual seed genuinely propagated into `plan_centralized()`'s ordering and the message bus's RNG (previously hardcoded to 42 regardless of the requested seed).

With only 4 scenarios where both `STOP_AND_WAIT` and the proposed system completed every run (`a_normal`, `b_intersection`, `e_blocked_aisle`, `h_negative_control`), the honest pooled result was **-23.68%** (unweighted -23.02%) — the proposed system was, at this point, genuinely *slower* than the naive baseline on the scenarios where a fair comparison was even possible. This was reported plainly as **NOT MET**, not smoothed over.

Stage 3 — a real fix, the gap more than halved (`benchmark_results_solution1/` , 2026-09-06 09:17, 981.4s)

Root cause traced precisely in `robot_agent/agent.py` : a robot that entered `WAITING` had no mechanism to notice its blocker had cleared — `_apply_action()` (the only code path that could flip status away from `WAITING`) only ran when `ConflictDetector.detect()` found a conflict *that same tick*; once the peer's path no longer contained the contested cell, nothing looked again, and the robot sat idle until the deadlock detector's 5-tick stall threshold forced an unnecessary full reroute. Fixed with `_waiting_condition_cleared()` — a genuinely conservative check requiring **all three** of: no fresh conflict this tick, no peer physically holding the next cell, and the *specific* blocking peer's raw reported position (not its self-labeled status) not being the next cell. That third condition was added only after the first, two-condition version caused a real, referee-verified collision in `d_deadlock` during this fix's own validation — R1 self-reported `MOVING` while, under one tick of message delay, still physically occupying the cell R2 was about to resume into. This is disclosed here deliberately: a genuine safety bug was introduced and caught before being reported as done, not swept past.

Result: `b_intersection` 16 → 11 ticks; pooled -23.68% → **-10.53%** (unweighted -9.13%). Still below target, reported as such.

Stage 4 — an instructive regression that was never reported as a final number (`benchmark_results_step1234/` , 2026-09-06 10:59, 1005.7s)

A separate fix, in the same session window, closed `f_task_reassignment`'s timeout (a genuine gap: nothing in `simulation/runner.py` had any mid-run task-reassignment logic at all — the idle backup robot R2 just sat there forever). Once `DECENTRALIZED_PROPOSED` could complete `f_task_reassignment`, the benchmark's own "is this scenario comparable" logic newly included it in the pool — but at this exact moment, `STOP_AND_WAIT` and `CENTRALIZED_RESERVATION` **still silently ignored** the scripted OFFLINE fault entirely (a separate, not-yet-found bug: they never called `agent.tick()`, the only code path that reads sensor telemetry). So `STOP_AND_WAIT` "completed" `f_task_reassignment` in 8 ticks by simply never noticing R1 had gone offline, while the proposed system genuinely detected the fault and handed the task off in 28 ticks — a -250% "improvement" for that one scenario, computed independently from the raw CSV (`stopAndWaitMeanMakespan: 8` , `decentralizedMeanMakespan: 28`) during this document's preparation. Pooled across the now-5 "comparable" scenarios, this dragged the number to **-52.17%** (unweighted -57.30%).

This number was never presented as a milestone result — it is reconstructed here specifically because it is the concrete evidence that made the next fix necessary and obviously correct, not a hypothetical. It is a textbook example of exactly the kind of methodology trap this project spent real effort finding and closing.

Stage 5 — fixing the baselines' fairness, and splitting into categories (`benchmark_results_part1234/` , 2026-09-06 12:19, 1422.0s)

Two fixes, same session: (a) `STOP_AND_WAIT` / `CENTRALIZED_RESERVATION` now genuinely observe the same scripted events the proposed system does — a new `_apply_sensor_telemetry()` helper (reusing `RobotAgent._apply_telemetry()` verbatim, granting no new capability) and a blocked-cell check, so both baselines now correctly *time out* on `e_blocked_aisle` / `f_task_reassignment` instead of silently

succeeding by ignoring the fault; (b) the benchmark was restructured into the 5 categories above, so the $\geq 20\%$ claim is computed *exclusively* from genuine overlapping-path contention scenarios, never diluted by negative controls or resilience scenarios that were never testing the same thing.

Result: Category 1 (at this point, `b_intersection` alone was comparable) pooled to **-22.22%**. This is *not* a regression from -10.53% — it is a more honestly scoped number, computed only from scenarios that actually test the claim, with every baseline now genuinely facing the identical events.

Stage 6 — `i_parallel_aisles` and the current, final result (`benchmark_results_i_parallel_aisles/`, 2026-09-07 00:33, 1568.8s, 270 runs)

Why `b_intersection` alone could never reach 20%, proven, not just observed. `b_intersection`'s two robots each travel exactly 8 cells (Manhattan distance) to a single shared intersection cell that both routes must pass through — there is no alternate path. `STOP_AND_WAIT` already achieves 9 ticks: 8 ticks of movement plus exactly one tick lost by whichever robot yields at the one unavoidable shared cell. Since only one robot can ever occupy that cell at a time and there is no second route to spread the pair across, **no algorithm, however sophisticated, can complete both robots in fewer than 9 ticks** — `STOP_AND_WAIT` is already at the scenario's true optimum. This is why `DECENTRALIZED_PROPOSED`'s 11 ticks (-22.22%, the same figure Stage 5 above pools as Category 1's entire result at that point, since `b_intersection` was its only comparable scenario then) is not a defect to keep chasing: there is no 20%-sized gap to close here, full stop.

`i_parallel_aisles` was built, and pre-registered, specifically to give a real algorithm route-diversity headroom that a single-bottleneck scenario structurally cannot. Its geometry (three parallel 8-cell aisles, entrance-distance-tied, one 4-tick-shorter "direct" middle lane and two tied "detour" outer lanes) was verified analytically with the real planner *before* any benchmark run, not tuned afterward to fit a target — this document independently confirmed the geometric claims by reading the scenario file's own worked distance arithmetic and the map-builder function directly.

A real, unsafe experiment was tried and rejected first, not silently skipped: a congestion-aware planning term (`GridAStarPlanner.congestion_counts`, weight 0.5 per claiming peer) plus a first-tick route-reconsideration in `RobotAgent`, intended to let a robot proactively prefer an uncontested aisle. Isolated A/B testing found it produced **44 referee-verified collisions per run and 0/10 completions**, a genuine deadlock livelock traced to a circular wait chain in the priority-resolution/replan interaction — reproduced twice, independent of an unrelated `robotId`-ordering bug fixed in the same scenario file during its own validation. Per the project's own standing hard safety gate (zero collisions, no exceptions), it was **fully reverted** — confirmed by `grep` finding zero remaining trace of `CONGESTION_PENALTY` / `congestion_counts` anywhere in `robot_agent/` or `simulation/`.

Having reverted that, the team then found that the **existing, already-shipped, completely unmodified** `DECENTRALIZED_PROPOSED` algorithm already met the target on this scenario: **STOP_AND_WAIT 23 ticks, DECENTRALIZED_PROPOSED 18 ticks, $(23-18)/23 \times 100 = 21.74\%$** — independently re-verified from the raw CSV during this document's preparation, identical across all 10 seeds, 0 collisions. The margin was traced to the existing peer-intent-based queuing logic handling six same-direction robots more efficiently than `STOP_AND_WAIT`'s simpler reactive check — **not** from route diversity being exploited: a direct check found 0 of 6 robots in either mode ever actually used an outer aisle; every robot funnels onto the middle lane in both modes, because `GridAStarPlanner`'s

deterministic tie-break was separately verified (a throwaway 2-aisle pre-check, before this scenario was built) to fully collapse tied routes onto one lane rather than self-diversifying.

Current, final, authoritative result: 270 runs (9 scenarios × 3 modes × 10 seeds), independently re-verified from `raw_results.csv` during this document's preparation: **0 collisions in all 270 rows, 0 rows with an error.** Category 1 pooled (`b_intersection` + `i_parallel_aisles` together) = **9.38%**, still below target — expected, since `b_intersection`'s proven floor necessarily drags any pool containing it downward. The scenario-specific reading — `i_parallel_aisles` : **21.74%, meets the ≥20% target** — is the one to cite, and it must always be cited with the scenario name attached, never as an overall system claim.

The independent test-change audit

A separate, adversarial self-audit of the test-file edits made to accommodate the 9th scenario found the edit set was actually **5 test-function changes, not the 3 first reported** — a real finding about the team's own reporting accuracy, disclosed rather than minimized. Of the 5: three were harmless structural maintenance (a literal scenario-count update in three places), one was a brand-new test with no prior baseline to weaken, and one was a genuine redefinition of Category 1's expected membership — traced and confirmed to match a pre-written design brief authored *before* the 9th scenario existed, not invented after seeing favorable results. No assertion was loosened in any of the 5; nothing touching collision counts, the referee, `b_intersection`'s own values, or the improvement-percentage calculation itself was touched by any of them. The 21.74% figure was confirmed to have been produced by a standalone script *before* any of the 5 test edits were made, and reproduced unchanged by the full benchmark run afterward — i.e., the test edits and the headline number are demonstrably decoupled.

The live-demo coordinate fix, and why it does not touch the benchmark

Separately, a real gap was found in live full-stack testing: 8 of the 9 scenario files hardcode start/goal/event coordinates that are only valid against the *benchmark's own 20×20 fixture map* (`build_benchmark_warehouse_map()`) — never against the real, denser `demo_map.py` (20×15) that a live Java-driven run actually falls back to. This had never been caught before because every test and every benchmark run always passes its own explicit fixture map, so `get_demo_map()`'s real output was simply never exercised by anything except the live path. Fixed with a `using_real_demo_map = warehouse_map is None` branch in each of the 8 scenario files (verified directly in this document's preparation, reading all 8): live-only coordinates on that branch, the exact original benchmark/test literals preserved byte-for-byte on the other. **This is why it cannot have touched any benchmark number:** every benchmark run always supplies its own explicit map, so it always takes the untouched branch. `i_parallel_aisles` was not touched at all — it supplies its own dedicated map and ignores any externally-passed one, by design, on both paths.

One real, disclosed limitation surfaced during that same live-testing pass: `g_high_load`'s original 3-way *crossing* geometry (two horizontal pairs plus one vertical pair crossing both) does not reliably complete against the real `demo_map.py` — root-caused empirically to a length effect on long head-on spans and a specific vertical/row-14 crossing interaction in the existing, deliberately-untouched coordination logic. The live version instead demonstrates three independent simultaneous pairwise

conflicts (still 6 robots, still real negotiation, still 0 collisions) rather than the originally-intended 3-way crossing — a genuine scope reduction for the live demo specifically, not hidden, and not applicable to the benchmark's own fixture-map numbers above, which are unaffected.

Judge Q&A Preparation

Every answer below was checked against the verified facts in the Architecture Walkthrough, Architecture Decisions, and Benchmark Journey chapters — not transcribed from a draft assumption. Cross-references point back to those chapters for the full-depth version if a judge wants more than the short answer.

Q1. Your backend assigns all tasks centrally — isn't that centralized, not decentralized?

Yes, deliberately — we say this up front rather than being defensive about it. Task assignment (which robot does which pickup/delivery) is centralized in the Java backend by design; the decentralization claim is specifically about *movement-level* conflict and deadlock resolution, which is verified — not just claimed — to run with no central per-tick arbiter (see the Architecture Walkthrough, Flow B: every conflict decision is a function of one robot's own state plus messages it received, and

`ConflictDetector` / `ConflictResolver` / `DeadlockDetector` 's actual method signatures never accept a full robot roster). Real commercial AMR fleets (Amazon Robotics, Locus, Geek+, OTTO) also centralize task assignment — the argument for decentralizing movement specifically is resilience, not latency, and that argument simply doesn't apply to task assignment, which can be wrong and cheaply reassigned with no safety cost.

Follow-up: So what actually happens if the Java backend dies mid-run? Every collision/deadlock/makespan number in our final benchmark was produced with the backend URL pointed at a deliberately unreachable port — movement coordination never touched it, and the result was still 0/270 collisions. Robots keep planning, communicating over their own bus, and avoiding each other with no backend at all; only task *assignment* and the dashboard would be affected.

Q2. Why is there no machine learning in a project titled "Edge-AI"?

The category is Software with no physical hardware requirement, and the project's own documented position is that decentralized, local decision-making on each robot already satisfies "edge intelligence" — an on-device neural network was never a stated requirement of the problem statement's title. The actual "intelligence" here is the local conflict/priority reasoning each robot does with only its own state and peer messages.

Follow-up: Would a learned conflict predictor actually improve this system? Honestly, probably not much, and possibly negatively. Our deterministic priority rules already achieve 0/270 referee-verified collisions; a learned component would introduce nondeterminism that would undermine the seed-reproducibility our own benchmark journey relied on heavily to trust its own numbers (see the Benchmark Journey chapter). The real headroom we found (`i_parallel_aisles`) was geometric — route diversity — not decision-quality, which a smarter predictor wouldn't have addressed either.

Q3. Your benchmark result changed from +31.55% to 21.74% over the project's history — why should this final number be trusted?

Because every swing has a specific, disclosed, traceable cause, not an unexplained drift. +31.55% was computed under a methodology bug — timed-out runs were averaged in as if they'd completed exactly at the deadline, which inflated the naive baseline's apparent slowness and therefore our apparent improvement. Fixing that produced an honest **negative** number first (-23.68%), which we reported as a genuine shortfall, not hidden. From there, a real algorithm fix (-10.53%), a fairness fix to the baselines' own event-handling (-22.22%, correctly scoped to genuine overlapping-path scenarios only), and finally a new scenario purpose-built to test genuine route-diversity headroom (21.74%, meeting the target) — each step is a documented, re-derivable number, and I independently re-verified the final one directly from the raw per-seed CSV, not just a generated report, in preparing this document.

Follow-up: How do we know another bug won't change this number again? We can't promise zero future bugs — no honest team can. What we can point to is the track record: every correction so far was found by our *own* internal audits, not by an external party catching us, and every correction was disclosed with the old number kept visible, never quietly replaced. That process, not a guarantee of perfection, is what should carry credibility here.

Q4. Isn't the `i_parallel_aisles` scenario just built to make your algorithm look good?

Its geometry (three parallel 8-cell aisles, provably tied in entrance distance) was verified analytically with the real path planner *before any benchmark run*, not tuned after seeing a result — we can show the geometry's worked distance arithmetic directly in the scenario file. We also kept `b_intersection` in the reported categories despite it *disproving* the target is universally reachable (its makespan has a proven hard floor equal to `STOP_AND_WAIT`'s own optimum — see the Benchmark Journey chapter), and we fully reverted a separate, more aggressive optimization attempt specifically because it produced collisions, even though it might have pushed the number further in our favor.

Follow-up: If the geometry was designed with route-diversity headroom in mind, isn't targeting that goal itself a form of cherry-picking? Testing for a capability the scenario is explicitly designed to expose is standard MAPF benchmark practice (parallel-corridor scenarios are a recognized pattern in the literature), not cherry-picking a lucky result — cherry-picking would be discarding scenarios that came out unfavorably, which we didn't do (`b_intersection`'s unfavorable result is reported alongside, not hidden). Separately: the margin that actually produced 21.74% came from queuing efficiency, not the route-diversity mechanism the scenario was built to test — a real, unplanned, and disclosed finding, which is the opposite of the result being engineered.

Q5. You mention an optimization that caused 44 collisions per run — why should we trust your other safety claims?

Because that optimization was caught by our own validation process, rejected outright per a standing hard safety gate (zero collisions, no exceptions), and fully reverted before it was ever reported as a

shipped result — confirmed by grepping the codebase for zero remaining trace. That is the safety process working exactly as intended, not a near-miss that slipped through. Every collision-safety number in this document (0/270) is about the code that is actually running today, which never included that mechanism in any released state.

Follow-up: How do we know it's actually fully gone and not lurking somewhere? Two independent reasons: first, we confirmed by direct code search (`grep`) across the relevant modules that none of its identifiers remain. Second, even if some trace had been missed in decision-making code, our collision count is certified by `simulation/referee.py`, which is structurally independent of and imports nothing from the coordination logic that could have carried the bug — so the safety claim doesn't rely on trusting that the removal was perfect, only on the referee's own much simpler ground-truth position check.

Q6. What exactly is a "tick," and is this measuring anything close to real time?

A tick is one discrete simulation step; at most one grid cell of movement per robot per tick. Every number in our benchmark is a tick count, not a real-time duration. We do have a `LIVE` playback speed that throttles to roughly 2 ticks/second purely so a human can watch it, but every benchmark result was produced in `BATCH` mode with no artificial delay at all — the numbers describe algorithmic efficiency (how many discrete steps a plan takes), not wall-clock demo speed.

Follow-up: Doesn't discretizing to whole-cell ticks hide real-world timing effects like acceleration or turning radius? Yes, deliberately — this is a software-category simulation of the coordination *decision layer*, not a physical motion-planning or control system. The discrete-tick model is standard in the MAPF research this work is built on, and it's what makes every result exactly reproducible given a seed, which we relied on directly to validate our final numbers across 10 seeds.

Q7. Is your live demo running the exact same thing as your benchmark?

Same algorithm code and the same scenario logic, yes — but two different coordinate fixtures. The benchmark always runs against a purpose-built 20×20 test map, identical for every mode and every scenario. A live, judge-facing run through the real Java-driven path falls back to the actual, denser 20×15 production `demo_map.py` if no map is supplied — and 8 of the 9 scenario files needed a separate, disclosed live-only coordinate branch because their original hardcoded coordinates land on obstacles in that real map (they were only ever validated against the benchmark's own fixture). Both branches are exercised and both are verified working; we don't conflate them, and the benchmark numbers in this document all come from the fixture-map branch, unaffected by the live-coordinate fix.

Follow-up: Does that mean the live demo's numbers could differ from the benchmark's? Yes, and we don't claim otherwise — the live demo's tick counts (verified directly by running all three processes together) are close to but not identical to the benchmark's fixture-map numbers, because the real production map's geometry differs slightly from the test fixture. We only ever cite the fixture-map, 270-run result as the audited proof of the ≥20%/zero-collision claims — the live demo is a demonstration of the same system working, not a second data point for that claim.

Q8. Why is the backend written in Java when the rest of the system is Python?

Team skill distribution, plus a real technical fit: Spring Boot's REST/WebSocket/JPA tooling is mature for exactly the backend's actual job — task management, data ingestion, and a live-update facade — with comparatively little boilerplate, while Python's ecosystem suits the simulation/planning/coordination logic and rapid iteration (which mattered a great deal given how many methodology corrections the benchmark itself went through). See the Architecture Decisions chapter for the full reasoning, including how the resulting three-way contract-mirroring cost (Python dataclasses, Java DTOs, TypeScript interfaces) is deliberately managed, not accidental.

Follow-up: Doesn't that language split slow down development compared to one language everywhere?

There is a real, acknowledged cost — three mirrored copies of every shared data contract. We manage it with one canonical source (`shared/python/models.py`) and an explicit rule that the running Python code wins if a mirror ever disagrees, rather than pretending the cost doesn't exist.

Q9. What would you do differently with more time?

Several concrete, already-identified items, not a vague "polish everything": (a) two further candidate coordination fixes were designed but deliberately not implemented — deadlock stall-threshold tuning and ETA-based proactive waiting — because the $\geq 20\%$ target was met by other means first and we were explicitly told not to chase further optimizations once it was; (b) a live channel for Java's centralized OFFLINE-triggered task reassignment to actually redirect an *already-running* Python simulation doesn't exist yet — today, Python has its own separate, internal mid-run reassignment mechanism that produces the visible behavior, and Java's own reassignment decision has no path into a live run (see the Architecture Walkthrough); (c) `DeadlockDetector` uses a simplified N-tick stall heuristic rather than full wait-for-graph cycle detection, which is enough for the 2- and 3-robot deadlocks in our scenarios but not proven for arbitrary larger cycles; (d) the benchmark has still never been run directly against the real production `demo_map.py` — only the simpler 20×20 fixture, identically for every mode, which is a fair comparison but not the actual production geometry.

Follow-up: Of those, which would you prioritize first? The live Java→Python reassignment channel, because it's the one gap with direct judging/demo impact — everything else is an internal robustness or fidelity improvement that doesn't change what a judge can already see working today.

Q10. How is your system meaningfully different from what a real warehouse robot fleet (e.g., Amazon's) already does?

We don't claim to out-engineer Amazon Robotics — this is a software simulation demonstrating a specific coordination architecture and validating it with an independent referee, not a production fleet management system. Where we align with real practice: centralizing task assignment, as real fleets do, for the same reason (global visibility, cheap to correct). Where we specifically demonstrate something worth showing a judge: a movement-coordination layer that is provably resilient to the central server

disappearing entirely mid-operation, verified by actually killing it during every benchmark run, not merely asserted.

Follow-up: So what's actually novel here, if the architecture mirrors industry practice? Not the high-level hybrid split — that's standard. The specific, load-bearing evidence is the independent-referee-verified zero-collision result across nine deliberately varied scenarios including engineered deadlocks and dynamic faults, and the honest, multi-stage benchmark methodology that survived its own internal audits — the rigor of the *validation*, not a claim of a fundamentally new algorithm.

Q11. Why does `STOP_AND_WAIT` fail to complete in several of your scenarios — is that a fair baseline, or did you make it artificially weak?

It's a real, simple, reactive baseline — react to what's immediately visible, stop if blocked — not weakened for effect. The proof that it isn't sabotaged: `CENTRALIZED_RESERVATION`, our *other* baseline, completes those exact same scenarios (`c_narrow_aisle`, `d_deadlock`, `g_high_load`) successfully — so the scenarios are solvable, and `STOP_AND_WAIT`'s failures are a genuine structural limitation of naive reactive planning under real contention, not an artifact of us handicapping it.

Follow-up: Then why include `STOP_AND_WAIT` at all if it's a weaker baseline? Because "does the proposed system complete work a naive approach structurally cannot" is itself a real, reportable capability finding (Category 2), distinct from and not blended into the $\geq 20\%$ speed claim, which is scoped only to scenarios where a fair head-to-head comparison is even possible.

Q12. How do you know your independent collision referee isn't just as buggy as everything else, giving you false confidence?

Two structural reasons, not just an assertion: first, `simulation/referee.py`'s `CollisionReferee` imports nothing from `collision_engine` at all — verified directly by reading its imports, which are only `shared.python.models.Position` — so a bug in the conflict-detection/resolution logic cannot also corrupt the thing checking for its failures; that would be circular, and we specifically avoided it. Second, its actual logic is intentionally much simpler than the detector it's independent of — a same-cell check and an edge-swap check on raw ground-truth positions, nothing else — which makes it far easier to read and trust correct in the first place than the multi-rule conflict/priority system it's certifying.

Follow-up: Could the referee itself have a subtle bug in that simpler logic? We can't claim mathematical proof of zero bugs in any code, including the referee. What we can say is that its logic was read and verified directly as part of preparing this document, it's short enough to audit in a few minutes, and it caught real collisions during earlier, pre-fix stages of this project (see the Benchmark Journey chapter) — it has a demonstrated track record of actually flagging problems, not just reporting a suspiciously clean zero from day one.

Q13. Why do you have 9 scenarios and 5 separate metric categories instead of one clean benchmark number?

Because a single blended number would conflate fundamentally different questions — speed, reliability, capability, and absolute safety are not the same axis, and averaging them together lets a strong result on one silently paper over a weak result on another (this is literally what happened at one stage of our own benchmark history — see the Benchmark Journey chapter's Stage 4 — and it's exactly the trap we built the category structure to close). Five separate, precisely-scoped categories is more honest, not more complicated for its own sake: a judge asking "is it fast?", "is it reliable under contention?", "does it handle dynamic failures?", and "is it safe?" gets four different, correctly-scoped answers instead of one number that can't actually answer any of them cleanly.

Follow-up: Doesn't that make it harder for a judge to quickly grasp the result? We think the opposite — one blended number that turned out to hide a real methodology bug earlier in this exact project (see Stage 4) is what's actually hard to trust quickly. Five short, plainly-labeled answers take about the same time to state and are each independently defensible.

Q14. If I added a 4th, 5th, 6th robot to your best-performing scenario, what would happen?

Honestly: we haven't measured that, and we won't invent a number for it. `i_parallel_aisles` is currently exactly 6 robots matched one-to-one against 6 distinct goal cells on a specific geometry; adding more robots would require either extending the staging row or queuing additional robots behind existing ones, and we have no benchmarked data beyond 6 robots on this scenario. We do have a scenario that already tests 6 concurrent robots under contention (`g_high_load`, three simultaneous head-to-head pairs, 0 collisions), which is the closest evidence we can honestly point to for fleet-scale behavior.

Follow-up: What would you predict, even without having measured it? We'd expect `STOP_AND_WAIT`'s relative disadvantage to grow, not shrink, as more robots share the one direct lane — more queuing for the naive reactive baseline — but we're explicitly flagging this as an unmeasured prediction, not a claimed result, because that's the standard this document holds every other number in it to.

Q15. What's the single biggest technical mistake your team made during this project, and how did you find it?

The +31.55% methodology bug (Stage 1 of the Benchmark Journey chapter): counting a timed-out run as if it had completed exactly at its deadline, which inflated the naive baseline's apparent slowness and therefore our own apparent improvement, to the point that an internal verdict function was reporting "VERIFIED" on a number that shouldn't have been trusted. It was found by a deliberate, strict internal audit of the benchmark's own methodology — not by an external reviewer or a judge catching it — which is exactly why we're comfortable stating it plainly here rather than hoping nobody asks.

Follow-up: Given that, how confident are you in the numbers presented today? Confident in the specific, narrow claims this document actually makes (0/270 collisions; 21.74% on `i_parallel_aisles` specifically) because each was independently re-derived from raw per-seed data during this

document's own preparation, not just trusted from a prior report — and explicitly not confident in, and therefore not claiming, anything broader than that (like a pooled/overall speedup), which is the whole point of scoping the claim as narrowly as the evidence actually supports.

Additional judge questions (team-authored, covering angles not already listed)

Q16. Your `DeadLockDetector` is a simple N-tick stall heuristic, not real wait-for-graph cycle detection — could it miss a genuine 3-or-more-robot circular deadlock?

Possibly, for an arbitrarily complex cycle — this is an explicitly acknowledged, documented limitation (`role/03_COORDINATION.md` §8.3 calls full cycle detection an optional, not-required enhancement). What it does reliably handle, verified by the benchmark, is the 2-robot and typical small-group deadlocks our scenarios actually engineer (`d_deadlock`), and the mutual-wait pairs inside `g_high_load`, each recovering cleanly with 0 collisions. We would not claim it handles an arbitrary N-robot circular wait we haven't specifically tested.

Follow-up: Isn't that a real gap for a "smart warehouse" claim? It's a real, disclosed scope limit, not a hidden one — the project's own role specification explicitly scoped full cycle detection out of the MVP as a Phase-4+ stretch, and every deadlock scenario we do claim to handle is benchmarked and referee-verified at 0 collisions, not just asserted.

Q17. You rejected a congestion-aware optimization because it caused collisions in one scenario — does that mean congestion-awareness is fundamentally incompatible with your safety guarantees, or was it just a flawed implementation?

We can only speak to what we actually tested: this specific implementation (a congestion-weighted planning term plus a first-tick route reconsideration) produced a circular wait chain in the priority-resolution/replan interaction. We have not built or tested a different, safer congestion-aware mechanism, so we can't honestly claim the general idea is impossible — only that this attempt was unsafe and was rejected rather than patched under time pressure.

Follow-up: Why not just fix the specific bug that caused the collisions instead of reverting entirely? Because the failure was traced to a genuine interaction between two subsystems (priority resolution and replanning), not an isolated one-line bug, and our standing rule is that a change touching safety-critical coordination logic doesn't ship without full validation passing — reverting to the last known-safe state and reporting the finding honestly was the correct call under that rule, not a shortcut.

Q18. Your `deadLockCount` metric reads 2 for one scenario and 4 for another — what does that number actually mean, precisely?

It's the number of genuine `DeadLockDetector.check()` activations — actual forced-yield recoveries — in a single run, counted by a pass-through wrapper around the real detector so the metric can't silently

drift from what the detector actually does (an earlier version of this metric was in fact disconnected from the real detector for a time — see the Benchmark Journey chapter's Stage 2 fix). `d_deadlock` (a single engineered 2-robot standoff) shows 2; `g_high_load` (three simultaneous pairs) shows 4 — consistent with roughly one activation per side of each genuine standoff.

Follow-up: Is a higher deadlockCount a bad sign? Not by itself — it means the scenario genuinely engineered contention severe enough to require the recovery mechanism, which is exactly what those scenarios are designed to do. The number that would actually be bad is a deadlock activation followed by a collision or a non-completion; every deadlock activation in our benchmark is followed by successful completion with 0 collisions.

Q19. Your resolver's final tie-break is lexicographic robot ID — isn't that arbitrary, and could it starve a particular robot over a long run?

It's arbitrary by design — a tie-break only ever fires after four more meaningful rules (task urgency, task priority, already-in-the-cell, rerouting cost) have all failed to distinguish the pair, so it's a last resort for genuine ties, not the primary mechanism. We haven't measured long-run fairness/starvation across many repeated tasks for the same robot pair — our scenarios are single-pass (one trip per robot), so starvation across repeated tasks is a real, open question we haven't tested and won't claim an answer to.

Follow-up: Would that matter in a real, continuously-operating warehouse? Potentially yes, over a long enough operating window with the same pair of robots repeatedly contesting the same cell — this would be a legitimate area for follow-up work; task priority (which we do model) would need to genuinely vary over time to avoid a robot with equally-low, never-changing priority always losing ties, and we haven't built or tested a mechanism for that.

Q20. The dashboard shows each robot's own negotiation decision separately — how do we know two robots aren't shown telling contradictory stories about the same conflict?

They can, in fact, look asymmetric on screen, and that's deliberate and accurate, not a bug: each `NEGOTIATION` event is one robot's own local resolver verdict (`CONTINUE` / `WAIT` / `YIELD` /etc.), read directly from that robot's own already-computed decision — there is no shared, arbitrated "the conflict was resolved as X" record anywhere in the system, because there genuinely is no central arbiter. If robot A shows `CONTINUE` and robot B shows `WAIT` for the same tick, that's two robots' independent, locally-correct readings of the same situation, not a data inconsistency — which is itself a faithful, honest visualization of what "decentralized" actually means, rather than a narrated single "verdict" that would misrepresent the architecture as centralized.

Follow-up: Could that ever look confusing or contradictory to someone watching who doesn't understand the architecture? Yes, which is why the dashboard's communication panel explicitly labels each line as "one robot's own local decision" rather than presenting a merged transcript — the confusion risk is real, and we address it with an explanatory label rather than smoothing the data into a false single narrative.

Q21. Your independent referee checks ground-truth positions — but doesn't it still rely on the same simulation state everything else produces? What if that underlying state itself is wrong?

That's a fair limit to be precise about: the referee is independent of the *decision-making* logic (`collision_engine`) — it cannot be fooled by a bug in `ConflictDetector` / `ConflictResolver` — but it does read the same ground-truth position variables the simulation's own tick loop maintains, so a bug in how a robot's *position itself* is updated (as opposed to how conflicts are decided) could in principle affect both. We don't have an *active, running* independent check on that narrower risk — being precise rather than overselling this: `planner/astar.py:validate_path()` (a path-validity checker — bounds, obstacle-overlap, tick-consistency) exists and is exercised in that module's own tests, but it is not called anywhere in the live or benchmark execution path, so it isn't actually mitigating this at runtime today. What we can point to instead is that the movement-update code path (`RobotAgent._advance_one_cell_if_clear()`) is a single, small, directly-read function — genuinely easy to audit by inspection — not a mitigation we're asking anyone to trust blindly, but also not a second independent system the way the collision referee is.

Q22. Why is task priority a fixed number hardcoded per scenario rather than dynamically computed from real urgency or deadlines?

MVP scope, stated honestly rather than dressed up: `Task.priority` (1–5) is a static field set at task creation, and our scenarios assign it directly rather than deriving it from a deadline/urgency model, because building a real urgency-scoring system was out of scope for validating the coordination layer itself, which is what this project's core claims are about.

Follow-up: Does that undermine the "priority-based" conflict resolution story? No — the resolver's priority rule genuinely works on whatever priority value it's given, tested and verified with real (if hardcoded) values; what's out of scope is *how* that value would be computed in a fuller system, which is a task-allocation-layer question, not a coordination-layer one, and task allocation is explicitly the centralized Java side's job, not this layer's.

Q23. Your live WebSocket feed can't replay history — what happens if a judge's laptop or the venue Wi-Fi drops the connection mid-demo?

The frontend auto-reconnects with exponential backoff (1s, up to 8s) and, on reconnect, immediately receives a fresh snapshot of current robot state — so the dashboard recovers on its own within a few seconds without restarting the simulation. What's genuinely lost is the *tick history* between disconnect and reconnect (there's no backend-side replay mechanism, a known, disclosed limitation) — a judge would see current state resume, not a rewind of what they missed.

Follow-up: Isn't that a real risk for a live judged demo? It's a real, disclosed limitation we plan for operationally rather than pretend doesn't exist: the actual coordination/collision-safety proof doesn't depend on the dashboard being connected at all (it's independently benchmarked offline), so a dropped dashboard connection during a live demo is a presentation inconvenience, not a correctness or safety issue — recoverable by simply reconnecting or re-running the scenario.

Q24. Does this approach scale — what happens with, say, 50 robots instead of 6?

We don't know, and we're not going to claim we do — every result in this document was produced with at most 6 concurrent robots (`g_high_load`, `i_parallel_aisles`), which was the project's own stated demo-scale target (`SIH_26123_Project_Overview.md` §1: "4–6 AMRs for demo purposes"). The per-tick cost of `ConflictDetector.detect()` grows with the number of peer intents compared per robot, so a much larger fleet would need real profiling before any scaling claim could be made honestly — that's explicitly unmeasured, not something we're asserting works or doesn't.

Glossary

API (Application Programming Interface) — a defined set of requests one piece of software can make to another. *In this project:* the Java backend exposes an API like `GET /api/robots` that the dashboard calls to get the current list of robots.

REST (Representational State Transfer) — a common style of API where you interact with named "resources" (like "a robot" or "a task") using standard HTTP verbs (GET to read, POST to create/trigger an action). *In this project:* `POST /api/simulation/control` with `{action: "start", ...}` is a REST call that tells the backend to start a scenario.

WebSocket — a persistent, two-way connection between a client and server that lets the server push data to the client the instant it happens, instead of the client having to keep asking "anything new?" (polling). *In this project:* `/ws/live` is the WebSocket the dashboard opens once, and the backend then pushes a new frame of robot positions down it every simulation tick, live.

Tick — one discrete step of the simulation clock. Every robot moves at most one grid cell per tick. *In this project:* `i_parallel_aisles` completing in "18 ticks" means it took 18 discrete simulation steps for the last robot to reach its goal — not 18 seconds.

A* search — a classic pathfinding algorithm that finds the shortest route on a grid or graph by always exploring the most promising next step first (using an estimate of remaining distance to the goal). *In this project:* the live simulation's `GridAStarPlanner.plan()` (`simulation/runner.py`) runs A* on the warehouse grid to compute each robot's route, using straight-line grid distance (Manhattan distance) as its "how far is left" estimate — `planner/astar.py::Planner` implements the same idea with a fuller feature set (dedicated re-routing, stricter error handling) and is real, tested code, though the live/benchmark path runs the `simulation/runner.py` version specifically (see the Path Planning role chapter).

MAPF (Multi-Agent Pathfinding) — the general research problem of finding routes for multiple agents that don't collide with each other, which is much harder than planning one agent's route alone because agents' plans can conflict with each other in space *and* time. *In this project:* the whole coordination layer — conflict detection, priority resolution, deadlock recovery — exists because standard single-robot A* has no idea another robot exists; MAPF techniques are what let multiple robots' independently-computed routes coexist safely.

Conflict — a *predicted* future collision, detected in advance from robots' broadcast intended paths, before it would actually happen. *In this project:* `ConflictDetector.detect()` sees that Robot A and Robot B both plan to be in cell (12,7) at tick 9, and raises a `Conflict` — nobody has actually collided yet.

Deadlock — a situation where robots are stuck waiting on each other with no one making progress, because each is (directly or indirectly) blocked by the other. *In this project:* two robots facing each other in a narrow corridor, each waiting for the other to move first — `d_deadlock` engineers exactly this, and `DeadlockDetector` breaks the stall by forcing the lower-priority robot to yield and reroute.

Decentralized vs. centralized coordination — decentralized means each agent decides for itself, using only what it directly knows or has been told; centralized means one authority decides on everyone's behalf. *In this project:* task assignment is centralized (the Java backend decides who does which delivery); movement/conflict decisions are decentralized (each `RobotAgent` decides its own next move using only its own state plus messages it received — verified by checking that none of the coordination functions ever receive a full robot list).

Referee / independent verification — checking a system's own safety claim using a *separate* piece of logic that doesn't share code with the thing being checked, so a bug in the main system can't also hide itself from the check. *In this project:* `simulation/referee.py`'s `CollisionReferee` never imports anything from `collision_engine` (the conflict-avoidance logic) — it independently re-checks raw robot positions for overlaps, so "zero collisions" is a verified fact, not a claim the same buggy code is grading its own homework on.

Seed / reproducibility — a starting number fed into a random-number generator so that anything "random" in a run (like the order robots are planned in) comes out exactly the same way every time you use that same number. *In this project:* running `i_parallel_aisles` with seed 1 ten separate times, or running it once each with seeds 1 through 10, both produce the exact same 18-tick result every time — this is what let us trust the final benchmark number wasn't a lucky one-off.

Makespan — the total time (in this project, ticks) from the start of a run until the *last* robot finishes — the standard way multi-agent pathfinding work measures "how long did the whole job take," since a fleet isn't done until every robot is. *In this project:* `i_parallel_aisles`'s "makespan" of 18 ticks (decentralized) vs. 23 ticks (stop-and-wait) is what the 21.74% improvement is computed from.

Completion rate vs. throughput percentage — a completion rate answers "did it finish at all?" (a fraction, like 100% or 0%); a throughput/speed percentage answers "how much faster did it finish, given that both did finish?" — these are different questions and must never be blended into one number. *In this project:* `STOP_AND_WAIT` has a 0% completion rate on `d_deadlock` (it never finishes within the deadline at all) — there is no "percent slower" number to compute there, because there's nothing to compare against; on `i_parallel_aisles`, both modes *do* finish, so a genuine 21.74% throughput percentage can be computed. Reporting "0% completion" as if it were "100% slower" would be a fabricated number, and this project deliberately keeps the two kinds of claim in separate categories so nobody on the team ever does that by accident.

PART II

Role-Specific Chapters

One chapter per module. Each is compulsory for the member who owns that module and optional-but-encouraged reading for everyone else.

Role Chapter: Path Planning

1. Fundamentals

Grid representation. A warehouse floor is modeled as a 2D grid: `gridWidth × gridHeight` cells, each either walkable or an obstacle. This is the simplest possible spatial model that still captures everything that matters for this problem — where a robot can and can't be.

A* search, from scratch. A* is a way to find the shortest route between two points on a grid (or graph) without checking every possible path. It keeps a priority queue of candidate partial paths, always expanding the one that looks most promising next — "most promising" meaning *cost so far* (`g`) plus an *estimate of remaining cost* (`h`, the heuristic) is smallest. If the heuristic never overestimates the true remaining distance ("admissible"), A* is guaranteed to find the shortest path. On a grid where you can only move up/down/left/right (no diagonals), **Manhattan distance** ($|x_1 - x_2| + |y_1 - y_2|$) is exactly this kind of safe estimate — it can never overestimate, because you can never need fewer moves than the straight-line grid distance.

Space-time A* / windowed cooperative replanning. Plain A* only knows about *space* — it has no idea a cell might be occupied by someone else at a *specific time*. Space-time A* extends the search state from `(x, y)` to `(x, y, tick)`, so it can be told "cell (5,5) is fine at tick 3 but forbidden at tick 7" and route around exactly that. This is the actual mechanism that lets one robot's route avoid another robot's *announced* future path without a central authority ever computing both robots' paths together — each robot just treats the other's broadcast intent as temporary, time-stamped obstacles. This general technique is a recognized MAPF (multi-agent pathfinding) method related to **Cooperative A*** and **Windowed Hierarchical Cooperative A* (WHCA*)** — naming it this way to a judge signals real familiarity with the research area rather than presenting an ad-hoc invention.

Centralized reservation-table planning, for contrast: instead of each agent planning independently and reacting to others, one planner plans every robot's route *sequentially*, and after each robot's path is chosen, every space-time cell it will occupy is "reserved" so the next robot's search simply can't route through them. This guarantees zero conflicts *by construction* for the robots planned this way — the tradeoff is that it needs one authority with full visibility of every robot's plan, which is precisely the resilience weakness a decentralized system is trying to avoid for movement decisions specifically.

2. This project's specific implementation

`planner/warehouse_map.py:build_warehouse_map()` constructs a validated `WarehouseMap` (`shared/python/models.py`) from raw grid parameters, raising `InvalidMapError` at build time (not later, silently) if any obstacle/choke/pickup/drop point is out of bounds or a pickup/drop/choke point coincides with an obstacle cell.

`simulation/warehouse/demo_map.py::get_demo_map()` builds the actual 20×15 demo warehouse: two vertical single-file aisles (columns 2 and 12, "Choke A"/"Choke B") through two shelf blocks, plus a horizontal cross-aisle at row 7 — read directly from the module's own ASCII-art layout comment and obstacle-builder function. This isn't an incidental map; it's a first-class design requirement (`role/01_PATH_PLANNING.md` §8.1: "a sparse map with rare conflicts makes the whole system's improvement claim look weak").

`planner/astar.py::Planner` is this module's own real, tested, standalone-runnable per-robot planner. `plan(start, goal, blocked_cells, avoid_intervals, start_tick, robot_id)` runs the A* described above — space-time when `avoid_intervals` (peer `(cell, tick)` claims) is non-empty, plain grid A* otherwise — and returns a `RobotPath` with `.tick` set on every waypoint. It validates start/goal aren't on an obstacle (`InvalidMapError`) and raises `PlanningFailedError` (never crashes) if no path exists. `replan(current_path, new_blocked_cells, current_tick)` is a dedicated re-routing method: it locates the robot's current position along its existing path, merges the new blocked cells into the map's mutable `blockedCells`, and re-plans from there to the original goal. `validate_path()` is a standalone utility checking a `RobotPath`'s waypoints for bounds, obstacle/blocked-cell overlap, adjacency, and tick-monotonicity.

A precise, verified fact worth stating plainly rather than glossing over: this `Planner` class — including its `.replan()` method, `PlanningFailedError` / `InvalidMapError`, and `validate_path()` — is exercised in this module's own unit tests and its standalone smoke-test (`python -m planner.astar`), but it is **not** what `simulation/runner.py` actually instantiates for the live `DECENTRALIZED_PROPOSED` / `STOP_AND_WAIT` execution path. `simulation/runner.py` defines its own separate `GridAStarPlanner` class directly in that file — a 4-connected grid A* supporting the identical `avoid_intervals` mechanism, explicitly commented in its own source as "conforming to Member 1's Planner interface" — and that is the class every live `RobotAgent` is actually constructed with. Confirmed by direct grep: `simulation/runner.py` contains zero imports from `planner.astar`, and `robot_agent/agent.py` never calls `.replan()` on anything — a scripted `AISLE_BLOCKED` event instead works by setting `_replan_requested = True` on each agent, which causes the *next* tick's `_needs_replan()` check to call `self.planner.plan()` (the runner's `GridAStarPlanner`, with the now-updated blocked-cell list) fresh, not this module's dedicated `.replan()` method. The one exception is `planner/centralized_baseline.py::plan_centralized()` — it genuinely is imported by `simulation/runner.py` and used directly for the `CENTRALIZED_RESERVATION` mode, unmodified.

`planner/centralized_baseline.py::plan_centralized(start_goals, warehouse_map, seed)` is the `CENTRALIZED_RESERVATION` comparison-mode planner: `random.Random(seed).shuffle()` on the robot list decides planning order (deterministic per seed), then each robot is planned sequentially against one shared `reservation: dict[(x,y,tick) -> robotId]` table via `_astar_reserved()`. This function rejects both a vertex conflict (the target cell/tick already reserved) *and* an edge/swap conflict — the classic MAPF case where two robots' paths would cross the same edge in opposite directions between consecutive ticks, sharing no single cell at any single tick and therefore invisible to a vertex-only check. This edge-conflict rejection (lines 246–248 of the file) was added after a real, referee-verified collision was found in an earlier stage of the project (see the Benchmark Journey chapter) — the module's docstring was corrected at the same time to state the true guarantee ("zero vertex AND zero edge/swap collisions"), not the original, narrower claim.

3. Alternatives considered and rejected

- **Graph representation instead of a grid** — rejected: a grid is simpler to reason about and renders directly onto the dashboard's SVG canvas with no separate layout step; a warehouse's aisle structure is naturally grid-like anyway.
- **A full centralized reservation table as the *only* planning mode** — rejected as the default: it works (and is offered as the `CENTRALIZED_RESERVATION` comparison baseline specifically because it's realistic and strong, not a strawman), but making it the *only* mode would remove the entire resilience argument the project is built around — no reservation table can be computed without one authority seeing every robot's plan.
- **8-connected movement (with diagonals)** — rejected in favor of 4-connected (`_MOVES = [(1, 0), (-1, 0), (0, 1), (0, -1)]`): diagonals complicate collision checking (a diagonal move can "cut through" a shared corner two other robots are swapping through) for a benefit the aisle-based warehouse layout doesn't need.
- **Full wait-for-graph-aware planning inside the planner itself** — rejected: deadlock reasoning belongs to Coordination (Member 3), not Path Planning; the planner's job is to answer "give me a route avoiding these named obstacles," and it does that whether the caller is avoiding a wall or another robot's claimed cell — keeping that separation is what let Path Planning be built and tested standalone before Coordination existed at all.

4. How this role communicates with every other role

To/From	Mechanism	What crosses
→ Coordination (Member 3), for <code>CENTRALIZED_RESERVATION</code> only	direct function call	<code>plan_centralized()</code> 's returned <code>RobotPath</code> per robot — genuinely used by <code>simulation/runner.py</code> for that one mode
→ Coordination (Member 3), conceptually	interface contract, not a direct import	<code>simulation/runner.py</code> 's own <code>GridAStarPlanner</code> is built to the same <code>plan(start, goal, blocked_cells, avoid_intervals, start_tick)</code> shape this module's <code>Planner</code> defines, and every downstream consumer (<code>ConflictDetector</code> , <code>RobotAgent</code> , broadcast intents) treats its <code>RobotPath</code> output identically — this module's <code>Planner / .replan()</code> themselves are exercised in this module's own tests, not by the live runner (see section 2's precise note)
→ Robot Communication (Member 2)	via <code>RobotAgent</code> , then <code>build_intent_message()</code>	Whichever planner instance is actually driving a robot, its <code>RobotPath</code>

To/From	Mechanism	What crosses
		waypoints become <code>plannedPath</code> in every broadcast intent message — this is literally what lets a peer's <code>ConflictDetector</code> see this robot's future route
← Robot Communication (Member 2)	<code>incoming</code> intents, read-only	Peers' broadcast <code>plannedPath</code> entries become <code>avoid_intervals</code> for windowed replanning — never mutated, only read
→ Simulation Runner (Member 3)	direct function call	<code>WarehouseMap</code> (loaded once at run start) and <code>plan_centralized()</code> (the <code>CENTRALIZED_RESERVATION</code> mode's entire planning step)
← Simulation Scenarios (Member 3)	<code>WarehouseMap.blockedCells</code> mutation	A scripted <code> AISLE_BLOCKED </code> event appends to <code>blockedCells</code> and sets <code>_replan_requested</code> ; the <i>next</i> tick's <code>GridAStarPlanner.plan()</code> call reads this mutable list fresh, so it "sees" the block automatically
→ Dashboard (Member 4)	indirectly, via Backend	The same <code>WarehouseMap</code> this module builds is what the dashboard's <code>WarehouseCanvas.tsx</code> renders — obstacles, choke points, blocked cells all trace back to this module's output, relayed through Java's <code>WarehouseController</code>

5. Evolution from the original role MD

`role/01_PATH_PLANNING.md` 's original specification is, to a close degree, what was actually built — this is one of the more stable modules across the project's history. Specific points of alignment, verified directly: the exact function signatures specified in §9 (`build_warehouse_map` , `Planner.plan` / `.replan` , `plan_centralized`) match the real, current code almost exactly, including parameter names and the documented exception types (`PlanningFailedError` , `InvalidMapError`).

What genuinely evolved: (a) the original spec's Definition of Done (§17) says "≥2 deliberate choke points" — the real `demo_map.py` has **three** (columns 2, 12, and row 7), a strengthening beyond the stated minimum, not a shortfall; (b) `plan_centralized()` 's actual collision guarantee grew stronger than originally specified — the original docstring/spec only claimed vertex-conflict freedom, and edge/swap-conflict rejection was added later, as a direct, traceable fix after a real collision was found in benchmark testing (see the Benchmark Journey chapter); (c) the original spec's `list_scenarios()` - adjacent context assumed 8 scenarios total; a 9th (`i_parallel_aisles`) was added later in the project's

history and supplies its **own** dedicated `WarehouseMap`, deliberately bypassing this module's `demo_map.py` entirely — a scoped, disclosed exception to "everyone uses the one demo map," not an oversight (see its own file's docstring for the exact reasoning); (d) a genuine divergence worth this role's own owner knowing precisely, found by reading `simulation/runner.py` directly rather than assuming: the original spec's §9/§10 describe `RobotPath / plan()` flowing from this module into `RobotAgent.tick()` as if this module's own `Planner` class fills that role at runtime. What was actually built has `simulation/runner.py` (Member 3's file) implement its own separate, simpler `GridAStarPlanner` class for that purpose instead, built to the same interface shape but not an import of this module's code — this module's own `Planner / .replan()` are real and tested, but only exercised by this module's own tests and standalone smoke-test, not by the live or benchmark execution path. `plan_centralized()` is unaffected by this and is genuinely used as originally specified.

6. How this role contributes to deadlock avoidance and coordination safety

Path Planning doesn't decide *when* to yield or *who* yields — that's Coordination's job — but this role's *interface design* supplies the mechanism Coordination's safety guarantees actually depend on: `avoid_intervals`-aware space-time planning is what lets a robot route around a specific, time-stamped peer claim instead of only reacting after entering a cell, and re-planning around a newly-blocked cell (whether that cell is a scripted obstacle or a peer a robot has been told to avoid via `_temporary_avoid_cells`) is what `RobotAgent._force_yield()` ultimately depends on to give a rerouted robot somewhere to actually go — concretely, `_force_yield()` sets `_replan_requested`, which drives the *next* tick's fresh `plan()` call (via `simulation/runner.py`'s `GridAStarPlanner`, built to this module's own interface shape — see section 2's precise note on which concrete class actually executes this at runtime). A bug in either planner (a bad path that clips an obstacle, or a path that silently ignores an avoid-interval) would propagate directly into every downstream safety claim, which is why `role/01_PATH_PLANNING.md` itself calls this "the highest-leverage module to get right" — a standard this module's own `Planner`, at least, is held to and tested against directly.

Role Chapter: Robot Communication

1. Fundamentals

Publish/subscribe messaging, from scratch. In a pub/sub system, a sender doesn't address a message to one specific recipient it has a direct handle to — it *publishes* to a shared channel, and every interested party independently *receives* whatever was published since they last checked. This decouples senders from receivers: a robot broadcasting its intent doesn't need to know who else exists, how many peers there are, or when they'll read it.

Why peer-to-peer messaging is what makes "decentralized" real rather than just claimed. A system could have every "decision" made locally in code while still secretly reading a global shared array of every robot's true state — that would *look* decentralized without actually being it, because nothing stops one robot's logic from depending on information it wasn't actually given through an explicit channel. Building a real message-passing layer, and making every coordination decision go through `Transport.receive()` rather than a shared object, is what makes the claim checkable rather than just asserted: you can point at the function signatures and show they never take a global robot registry.

Fault injection as a feature, not a bug. In real deployments, messages can be lost (a dropped Wi-Fi packet) or delayed (network congestion). Modeling this deliberately — a configurable drop rate and delay — turns "what if a robot doesn't hear about its neighbor in time" from an untested edge case into a scenario you can actually run and watch the system either handle gracefully or fail on.

2. This project's specific implementation

`robot_agent/communication/transport.py::Transport` is an abstract base class with exactly three methods: `send(to, msg)`, `broadcast(msg)`, `receive(robot_id) -> list[dict]`. Every caller in the codebase depends only on this interface.

`InProcessBus` is the concrete, default implementation — an in-memory, per-robot `deque` (`_queues: dict[robot_id -> deque]`), explicitly documented as designed for a single-threaded tick loop (no locking; a comment states plainly that thread-safe collections would be needed if the simulation were ever made concurrent — an honest, stated assumption rather than a silent one). `broadcast(msg)` enqueues to every registered robot **except** the sender (identified by `msg['robotId']`), raising `UnknownRobotError` if the sender itself isn't a registered ID — this is a real, tested failure mode, not a silent no-op.

`MessageFaultConfig(drop_rate, delay_ticks)` — validated strictly at construction (`ValueError` immediately for an out-of-range `drop_rate` or a negative `delay_ticks`, so a misconfiguration is caught at setup, not silently at message-delivery time three ticks later). A dropped message never

appears in `receive()` 's result at all; a delayed message is held in a separate `_delay_buffers` structure and only promoted into the ready queue once `advance_tick()` — called once per simulation tick by `simulation/runner.py` — has decremented its remaining delay to zero. This exact mechanism (`delay_ticks=1` on `d_deadlock`) is what let a real collision be found and fixed during the resume-on-clear validation (see the Benchmark Journey chapter): one tick of message delay was enough to hide that a peer reporting itself as `MOVING` was, in ground truth, still physically stuck.

`robot_agent/communication/messages.py::build_intent_message(state: RobotState) -> dict` extracts exactly `robotId`, `position`, `destination`, `plannedPath`, `currentTask`, `priority`, `battery`, `status`, `timestamp` from a `RobotState` — deliberately *not* a new named contract type, just a JSON-serializable subset of the existing one, which avoids duplicating field definitions. One documented compatibility note worth knowing precisely: `RobotState` (the frozen, canonical contract) has no `priority` field at all — `priority` is read via `getattr(state, 'task_priority', 1)`, a dynamically-set attribute, not a dataclass field, specifically so this module never had to modify the frozen shared contract to carry a value that conceptually belongs to `Task.priority`.

3. Alternatives considered and rejected

- **Real UDP or WebSocket sockets between separate robot processes, as the default** — rejected for the default demo path for a concrete, named risk: unfamiliar venue Wi-Fi at a live SIH demo is exactly the kind of thing that can fail for reasons that have nothing to do with the coordination logic being judged. `InProcessBus` is deterministic and immune to that specific risk. The `Transport` ABC exists precisely so a real network transport *could* be substituted later behind the identical interface (an isolated hardware-bonus segment) — designed for, never required, never built, since hardware is explicitly optional.
- **Thread-safe queues from the start (`queue.Queue`) instead of plain `deque`** — rejected as unnecessary complexity for the actual execution model: the simulation runs a single-threaded tick loop by design (documented explicitly in the module docstring), so locking overhead would have bought nothing; the note that switching would be needed *if* concurrency were ever added is an honest, forward-looking caveat, not a gap being hidden.
- **A dynamic robot-discovery protocol** — rejected: all robot IDs are known upfront from the scenario definition at simulation start, so a discovery handshake would solve a problem this project doesn't have.

4. How this role communicates with every other role

To/From	Mechanism	What crosses
← Coordination (Member 3, via <code>RobotAgent.tick()</code>)	direct function call	<code>RobotState</code> , the input to <code>build_intent_message()</code>
→ Coordination (Member 3)	<code>Transport.receive()</code> return value	delivered intent messages — the primary input to <code>ConflictDetector.detect()</code>

To/From	Mechanism	What crosses
→ Path Planning (Member 1)	delivered intent messages	peer <code>plannedPath</code> entries become <code>avoid_intervals</code> for windowed replanning
← Simulation Scenarios (Member 3)	constructor parameter	which <code>MessageFaultConfig</code> profile (drop rate, delay) a given scenario wants — e.g. <code>d_deadlock</code> 's <code>delay_ticks=1</code>
→ Simulation Runner (Member 3)	wrapping	<code>simulation/runner.py</code> wraps the real <code>InProcessBus</code> in a small local <code>_MessageCountingTransport</code> purely to tally <code>PerformanceMetric.messageCount</code> — pure delegation, no messaging logic of its own added

5. Evolution from the original role MD

`role/02_ROBOT_COMMUNICATION.md` 's specified interfaces (`Transport` , `MessageFaultConfig` , `InProcessBus` , `build_intent_message`) match the real, shipped code closely — this module's public shape is essentially exactly what was originally specified, including the constructor signatures. Two points worth naming precisely rather than glossing as "no change": (a) the original spec (§9) shows `InProcessBus.__init__(self, robot_ids, fault_config=None)` with no `rng` parameter; the real code adds an optional keyword-only `rng: random.Random | None` — a genuine, additive evolution, made necessary later in the project when real seed-propagation was fixed (a run's actual seed needed to reach the bus's fault-injection randomness, not just `plan centralized()` 's ordering — see the Benchmark Journey chapter's Stage 2). This is a backward-compatible extension (defaults to non-deterministic `random.Random()` if omitted), not a breaking change. (b) The intent message's `priority` field required the documented `getattr(state, 'task_priority', 1)` workaround described above — the original spec's §8.2 lists `priority` as part of the intent message's fields without flagging that `RobotState` itself doesn't carry one; this compatibility note was added directly in the shipped `messages.py` module docstring as the actual, discovered resolution.

6. How this role contributes to deadlock avoidance and coordination safety

This module doesn't decide anything about conflicts or deadlocks itself — but the entire deadlock-recovery story depends on it in a specific, precise way: `d_deadlock` 's engineered stall (see the Benchmark Journey chapter) exists *because* `MessageFaultConfig(delay_ticks=1)` genuinely delays a message, creating a real window where a peer's broadcast state lags its true physical state. Getting delay/drop semantics exactly right (a delayed message becomes visible after *exactly* `delay_ticks` calls to `advance_tick()` , never early, never late) is what makes that engineered scenario a faithful test of the resume-on-clear/deadlock-recovery logic rather than an accidentally-too-easy or too-hard one. A bug here — a message delivered one tick early, say — could silently make a deadlock scenario never actually deadlock, and the team would be testing nothing.

Role Chapter: Coordination

This is the largest role by design — it's the integration point of the entire robot-side system, and its module list touches almost every other role's boundary.

1. Fundamentals

Conflict detection, from scratch. Two robots that each independently plan a "correct" (obstacle-free) path can still collide with *each other*, because plain pathfinding has no concept of another agent existing. Detecting this in advance means comparing two robots' planned space-time routes for three specific patterns: (a) **same-cell** — both plan to be in the same cell at the same tick; (b) **crossing/edge-swap** — robot A moves from cell *u* to cell *v* exactly as robot B moves from *v* to *u*, so they pass through each other on the shared edge without ever technically sharing one cell at one tick; (c) **narrow-aisle head-on** — two robots approaching each other along a single-file corridor, which will produce a same-cell or edge conflict slightly further down their routes if nothing changes now.

Priority-based conflict resolution, from scratch. Once a conflict is detected, *something* has to give — one robot must yield, wait, reroute, or (rarely) have its task reassigned. Doing this deterministically (the same inputs always produce the same decision) is what makes multi-robot behavior debuggable and testable at all; a resolution policy that could go either way given identical state would make every replay non-reproducible.

Deadlock, and why simple stall-detection is a reasonable non-full solution. A true deadlock is a *cycle* of robots each waiting on the next (A waits for B, B waits for C, C waits for A) — detecting arbitrary cycles requires building and analyzing a "wait-for graph." For the 2-and-occasional-3-robot standoffs this project's scenarios actually engineer, a much simpler heuristic works just as well in practice: if a robot has made zero progress for *N* consecutive ticks while it's known to be waiting on a specific peer, treat that as a deadlock and force a resolution — no graph, no cycle analysis, a fraction of the code, and it is explicitly scoped this way rather than accidentally incomplete (`role/03_COORDINATION.md` §8.3 calls full cycle detection an optional future enhancement, not a requirement).

Why an independent referee matters. If the same logic that *tries* to avoid collisions is also the logic that *certifies* none happened, a bug that causes a real collision could plausibly also cause the certifying logic to miss it (they share the same blind spot). A referee that never imports or calls into the detection/resolution code, and instead directly compares raw ground-truth positions with much simpler logic, cannot share that blind spot — this is what makes "zero collisions" a claim a skeptical judge can actually be shown to trust, not just told to.

2. This project's specific implementation

`collision_engine/detection.py::ConflictDetector.detect(own_state, own_path, peer_intents)` compares this robot's own planned waypoints against every peer's broadcast `plannedPath`, running four checks in total (the three named above, plus a fourth: `_check_stationary_occupancy()`, which flags a peer's raw current position as a conflict whenever that peer's status marks it as not moving — added specifically because a stopped robot's occupied cell is popped off its own broadcast path the instant it arrives, so the path-based checks alone are structurally blind to "a peer is parked right here"). If multiple conflicts are found, the earliest-predicted-tick one wins, with severity as a tiebreak.

`collision_engine/resolution.py::ConflictResolver.resolve(conflict, own_state, peer_intents)` applies exactly five rules, in this strict order, to decide if `own_state`'s robot wins the conflict: (1) is one side an emergency/priority-5 task; (2) higher `Task.priority` (1–5); (3) is one side already physically inside the contested cell; (4) lower estimated rerouting cost (only used when both sides report an explicit cost); (5) lexicographically lower `robotId` as the final, arbitrary tie-break. The loser gets `WAIT` (same-cell/crossing conflicts), `YIELD` (narrow-aisle head-on — sets `_replan_requested` and adds the contested cell to `_temporary_avoid_cells`), or `REASSIGN_TASK` if it's already `OFFLINE`.

`collision_engine/deadlock.py::DeadlockDetector.check(robot_id, stall_ticks, waiting_on)` is exactly the simple heuristic described above: `return stall_ticks >= self.stall_threshold` (default 5) if `waiting_on` is not `None`, else `False`. The actual *asymmetric* recovery behavior a judge might expect from the detector itself (only the lower-priority robot reroutes) is implemented one layer up, in `RobotAgent._should_step_aside()` — this is a real, acknowledged documentation/layering nuance (see the Judge Q&A chapter and this project's own `CLAUDE.md` Known Bugs list), not a functional gap: the detector fires symmetrically for both sides of a stall, but only one side's `RobotAgent` actually reroutes.

`robot_agent/agent.py::RobotAgent.tick()` is the orchestration loop every other Python module plugs into — the full sequence is traced in the Architecture Walkthrough chapter's Flow B. Two specific, hard-won mechanisms live here, both documented in the source with the exact bug they fixed: `_waiting_condition_cleared()` (added so a `WAITING` robot can resume once its blocker genuinely clears, instead of always waiting out the full deadlock-detector stall threshold even when nothing is actually stopping it anymore — closing a real makespan gap on `b_intersection`), and the physical-occupancy guard inside `_advance_one_cell_if_clear()` (which refuses to move into a cell a stationary peer holds *regardless* of what the resolver decided — closing a real, referee-verified collision class found in `d_deadlock / g_high_load`).

`simulation/runner.py` is the FastAPI control server and tick driver (`/control/start`, `/control/stop`, `/control/status`), mode-switching between `DECENTRALIZED_PROPOSED` (real `RobotAgent.tick()` for every robot), `STOP_AND_WAIT` (a stripped-down reactive-occupancy step function), and `CENTRALIZED_RESERVATION` (one upfront `plan_centralized()` call, then execute the resulting schedule). It pushes telemetry/events every tick and a `PerformanceMetric` at run end, all resiliently (`_push_to_backend()` swallows failures — the simulation never halts because Java is unreachable).

`simulation/referee.py::CollisionReferee.check(all_positions)` — imports only `shared.python.models.Position`, nothing from `collision_engine`. Checks raw ground-truth `(x,y)`

positions for same-cell overlap and, comparing this tick's positions against last tick's, edge-swap overlap. This is the sole source of `PerformanceMetric.collissionCount`.

The 9 scenario files in `simulation/scenarios/` — (see the Benchmark Journey chapter for what each tests), each producing a `Scenario` dataclass (robots, warehouse map, scripted events, fault config, `max_ticks`).

3. Alternatives considered and rejected

- **Full wait-for-graph cycle detection instead of the N-tick stall heuristic** — explicitly scoped out as a Phase-4+ stretch; the simple heuristic handles every deadlock this project's scenarios actually engineer, verified by benchmark (0 collisions, successful recovery on every activation), and building full cycle detection would have spent real time on generality the demo scenarios don't exercise.
- **Letting the referee reuse `ConflictDetector`'s own logic** — rejected outright and explicitly forbidden by this role's own AI-coding boundary (`role/03_COORDINATION.md` §16): doing so would make "zero collisions" a circular claim, provable only by the same code that's supposed to be prevented from causing them.
- **A congestion-aware planning term to proactively spread robots across multiple routes** — attempted, found to produce 44 referee-verified collisions per run under a genuine circular-wait interaction with the priority/replan logic, and fully reverted per the project's standing zero-collision safety gate (see the Benchmark Journey chapter for the exact numbers and root cause).
- **Symmetric deadlock recovery (both sides of a standoff reroute at once)** — tried first, in fact, and found to cause both robots to dodge into the same neighboring cell on the same tick; `_should_step_aside()`'s asymmetric, priority-mirroring rule was adopted instead.

4. How this role communicates with every other role

To/From	Mechanism	What crosses
← Path Planning (Member 1)	direct call	<code>RobotPath</code> — primary input to conflict prediction and to actual movement
← Robot Communication (Member 2)	<code>Transport.receive()</code>	peer intent messages — primary input to <code>ConflictDetector</code>
← Edge (Member 5)	<code>SensorSource.read()</code>	<code>Telemetry</code> , folded into <code>RobotState</code> every tick; <code>sensorHealth=OFFLINE</code> drives the <code>RobotState.status=OFFLINE</code> transition
← Backend (Member 6)	<code>/control/start</code> request body	<code>taskAssignments</code> — overrides a scenario's hardcoded start/goal/priority per robot when real Java-side task data is supplied
→ Backend (Member 6)	REST push	<code>RobotState</code> batches (<code>/api/telemetry/batch</code>), <code>SimulationEvents</code> (<code>/api/events</code>),

To/From	Mechanism	What crosses
		<code>PerformanceMetric</code> (<code>/api/metrics</code>) — once per tick or once at run end respectively
→ Dashboard (Member 4)	indirectly, via Backend + WebSocket	everything the live view renders (robot positions/status, negotiation events, warehouse map) ultimately originates here

5. Evolution from the original role MD

`role/03_COORDINATION.md` on disk today is **not** a pristine original — it was itself edited once, narrowly, in a later documentation-sync pass (scenario count "8"→"9" in two places, since a 9th scenario was added after the original spec was written). The genuinely substantive evolution, beyond that count fix:

- **A 9th scenario, `i_parallel_aisles`, was added** — not in the original spec's scope at all, built specifically to give the decentralized algorithm genuine route-diversity headroom a single-bottleneck scenario (`b_intersection`) structurally cannot provide (see the Benchmark Journey chapter).
- **The `NEGOTIATION SimulationEvent` type did not exist in the original contract** at all (the original spec's §6/§9 list only `CONFLICT_DETECTED` / `DEADLOCK_DETECTED` / `REROUTE` as the coordination-produced event types) — it was added later, as an approved, purely observational addition for the dashboard's communication-visibility feature, reading attributes `RobotAgent` / `ConflictResolver` already compute rather than adding any new decision logic.
- **A documented, deliberate layering deviation from the original spec's prose:** the original spec's narrative (§8.3) implies the *detector itself* decides "only the lower-priority robot yields." What was actually built has the detector fire symmetrically and the asymmetric behavior live one layer up in `RobotAgent._should_step_aside()` — re-inspected directly and judged a real but low-severity documentation/layering issue, not a functional defect, since the resulting system behavior is correct either way.
- **A significant, unplanned safety hardening not present in the original spec at all:** `ConflictDetector._check_stationary_occupancy()` and `RobotAgent`'s physical-occupancy movement guard were both added later, after real, referee-verified collisions were found in benchmark testing — the original spec's conflict-type list (`SAME_CELL` , `CROSSING` , `NARROW_AISLE_HEADON`) had no equivalent of "a peer that has stopped moving still occupies its cell," and this gap produced actual collisions before it was found and closed.
- **Folder-naming:** the original spec refers to scenario/module paths consistent with what's on disk today (underscored names like `simulation/scenarios/`) — no hyphen-to-underscore rename was found in this module's own history during this document's preparation, unlike a change hinted at in this task's own background material; if such a rename happened, it predates or falls outside what this chapter's author could verify directly, and it is not claimed here as fact.

6. How this role contributes to deadlock avoidance and coordination safety

This is, by definition, the role whose entire job is deadlock avoidance and coordination safety — but it's worth being precise about which specific pieces do which specific job, since a judge may probe the boundary: `ConflictDetector` predicts trouble before it happens; `ConflictResolver` decides who yields, deterministically; `DeadlockDetector` catches the case where a resolved "yield" decision still doesn't produce progress; `RobotAgent`'s physical-occupancy guard is the last line of defense that holds even if every earlier decision was somehow wrong; and `CollisionReferee` is the outside check that none of the above can silently lie about its own success. The "zero collisions, verified" claim this whole project is built around rests on all five of these existing and being independently exercisable — not on any single one of them being perfect.

Role Chapter: Frontend

A note before this chapter starts, in the spirit of this document's own honesty standard: unlike the other five roles, no original role specification (`04_FRONTEND.md`) exists anywhere in this project's history — the `role/` folder on disk today contains files 01, 02, 03, 05, and 06 only, and `README.md` 's own documentation table references a `docs/04_FRONTEND.md` that has never existed at any point checked during this document's preparation. There is, genuinely, no "before" snapshot to compare against for this role's evolution section. Rather than invent one, this chapter builds its account entirely from the actual `dashboard/` source code and its own header comments, which — unusually for this codebase — directly narrate their own phase-by-phase history, verified by reading them directly.

1. Fundamentals

A single-page application (SPA), from scratch. Instead of the browser loading a new HTML page from the server for every view, a SPA loads once and then updates itself in place by re-rendering components as data changes — this is what lets a live warehouse map update every simulation tick without a page reload.

React and components. React organizes a UI as a tree of independent, reusable **components** — a function that takes some data (`props`) and returns what to display. `WarehouseCanvas` doesn't know or care how `RobotState` data arrived; it just renders whatever robot list it's handed. This separation is what let this dashboard be built mock-first (see below) — swap the data source, keep every component unchanged.

Hooks and state. A React "hook" is a function that lets a component hold onto changing data (`state`) and re-render automatically when it changes, and run side effects (like opening a network connection) tied to the component's lifecycle. This project's three central hooks (`useSimulationControl` , `useLiveFeed` , `useWarehouseMap`) each own one coherent slice of "what's currently true" and nothing else.

REST vs. WebSocket, from the client's side. A REST call (`fetch()`) is a one-shot question-and-answer — ask once, get one answer. A WebSocket is a standing connection the server can push into at any moment, unprompted — the only way to get genuinely live, tick-by-tick robot movement onto a dashboard without the browser having to constantly ask "anything new yet?"

SVG for the map. Scalable Vector Graphics lets a UI describe shapes (rectangles, circles, lines) declaratively in units that scale to any real size — this project renders the whole warehouse in a `viewBox="0 0 gridWidth gridHeight"` coordinate space, so one grid cell is always exactly one SVG unit regardless of whether the actual map is 20×15, 20×20, or `i_parallel_aisles` 's own 25×11 — no per-scenario pixel math anywhere.

2. This project's specific implementation

`dashboard/src/api/client.ts` — the real API client, talking only to the Java backend at a hardcoded `http://localhost:8080` (`ws://localhost:8080/ws/live` for the socket). Every function maps 1:1 onto a real backend endpoint: `fetchRobots()` → `GET /api/robots`, `fetchWarehouseMap()` → `GET /api/warehouse` (treating a `503` as a genuine "no map pushed yet" state, not an error to retry-loop on), `startSimulation()` / `stopSimulation()` → `POST /api/simulation/control`, `fetchSimulationStatus()` → `GET /api/simulation/status`, `fetchEvents()` → `GET /api/events`, and `connectLiveFeed()` opening the real `/ws/live` socket and parsing the `{type, tick, payload}` envelope.

`dashboard/src/types/contracts.ts` mirrors `shared/python/models.py` field-for-field (verified directly, side by side) — `RobotState`, `Task`, `Conflict`, `WarehouseMap`, `SimulationEvent` (including the `NEGOTIATION` payload shape), `PerformanceMetric`, plus the `LiveWsMessage` envelope union type and the simulation-control request/response shapes.

`dashboard/src/hooks/useSimulationControl.ts` owns scenario/mode/seed/speed selection and the real start/stop calls, plus a 1-second status poll — and separately tracks *two* different kinds of "is something down": whether the Java backend itself is reachable, versus whether Java is up but the Python engine underneath it is not (a real distinction discovered during integration testing: Java can return a clean `503` while staying fully alive).

`dashboard/src/hooks/useLiveFeed.ts` owns the WebSocket connection (with reconnect/backoff) and the client-side tick buffer that makes playback (`stepBack` / `stepForward` / pause/resume) possible at all — there is no backend history-replay mechanism, so this buffer *is* the only history that will ever exist for a given browser tab, built strictly from frames actually received while connected. It deduplicates events by their real `eventId` specifically because `simulation/runner.py` re-pushes its entire accumulated event list every tick (a real, discovered backend behavior this hook works around on the frontend side only, without asking for a backend change).

`dashboard/src/components/simulation/WarehouseCanvas.tsx` renders the real `WarehouseMap` (obstacles, choke points, dynamically blocked cells, pickup/drop points) plus real `RobotState` markers for whichever tick is currently displayed — nothing here is animated independently of actual received data.

`dashboard/src/components/events/CommunicationFeed.tsx` renders real `NEGOTIATION` events as **one robot's own local decision** per line — deliberately never a synthesized two-sided transcript, because there genuinely is no arbitrated, shared "the conflict resolved as X" record in a decentralized system (see Judge Q&A #20).

`dashboard/src/components/benchmark/BenchmarkView.tsx`, backed by `dashboard/src/data/benchmarkResult.ts` — a static, hand-transcribed mirror of the already-audited 270-run benchmark result (explicitly not fetched, not recomputed). The `i_parallel_aisles` 21.74% figure is never rendered without the scenario name attached in the same visual unit; the five categories are always rendered as separate sections; a mode that didn't complete a scenario shows the literal label `TIMEOUT / NOT COMPLETED` rather than a fabricated tick count; no blended "overall improvement" number is computed or displayed anywhere on the page.

Design system (`dashboard/src/styles/theme.css`) — deliberately flat, dark-first, high-contrast, explicitly modeled on industrial/SCADA control-room interfaces rather than a generic admin-dashboard template: no gradients, no glassmorphism, no decorative rounded cards. Status colors (`--status-idle` , `--status-waiting` , `--status-offline` , etc.) are defined once and reused identically across the map, the robot inspector, and the event timeline.

3. Alternatives considered and rejected

- **Polling instead of a WebSocket for live robot state** — rejected: polling every second (or faster) would either miss ticks between polls or hammer the backend; a push-based socket is the only way to get every real tick's frame without guessing an interval.
- **A generic component-library admin dashboard (cards, gradients, big charts)** — rejected in favor of the current industrial/SCADA visual language, deliberately, because the actual audience (judges assessing a real coordination system) benefits far more from a clear, legible map and status hierarchy than from decorative polish that doesn't represent real data.
- **Fetching the finalized benchmark result live from a backend endpoint** — rejected: no such endpoint exists, and building one would mean either re-running the 270-run benchmark on demand (far too slow for a live view) or having the backend store and serve a static artifact anyway — the static frontend data file achieves the same outcome (an accurate, audited display) without inventing backend surface area that adds risk for no benefit, and it keeps the "this is NOT live data" distinction structurally obvious rather than something a viewer has to be told.
- **Reading `/api/events` as the primary source of "what happened this run"** — rejected as the *primary* source (used only as an opt-in supplementary full log): the endpoint returns everything the backend has ever persisted, with no run/scenario tag on any event, discovered directly during integration testing — using it as the primary per-run source would silently mix in events from unrelated prior runs.

4. How this role communicates with every other role

The frontend talks to exactly one other role directly: **Backend (Member 6)**, over REST and WebSocket, as detailed in section 2 above. It never talks to Python (Coordination/Path Planning/Edge/Communication) directly — this is enforced by construction (there is no Python URL anywhere in the frontend's source) and is one of the project's frozen architectural rules. Everything the frontend displays about Path Planning's map, Coordination's conflict decisions, Communication's negotiation events, or Edge's fault/offline signals arrives exclusively as data already relayed through Java — the frontend has no independent channel to any of them, and does not need one.

5. Evolution (from the dashboard's own recorded history, since no original spec exists)

The dashboard's own source comments narrate a real, multi-phase build, verified directly by reading them: an early phase established a **mock-first** scaffold (`dashboard/src/api/mock.ts` , still present as unused scaffolding — schema-correct static fixtures, built so every component could be developed

before any backend existed), then a real `client.ts` was wired in behind the identical import point so components never had to change when the swap happened. Subsequent phases (each named directly in the components' own header comments as "Phase 4," "Phase 5," etc.) added, in order: the real app shell and scenario/mode/seed/speed controls wired to `POST /api/simulation/control`; the live warehouse view with real `/ws/live` buffering and tick playback; the communication/negotiation feed and event timeline (introduced alongside the `NEGOTIATION` event type becoming available — a genuinely new capability this role's own view had to be built to expose, not present from day one); and finally the Benchmark & Proof view, which for a time was an explicit, honestly-labeled "not yet implemented" empty state rather than being stubbed with fake content, until it was built out with the real, audited data described above. This progressive, phase-by-phase build — each phase visible in the code's own comments rather than reconstructed after the fact — is the closest equivalent this role has to the "evolution from an original spec" every other role can show directly.

6. How this role contributes to deadlock avoidance and coordination safety

The frontend makes no coordination decisions and cannot cause or prevent a collision — but it has a real, specific safety-adjacent obligation: **never imply a guarantee the system hasn't actually earned**, and never fabricate data to smooth over a gap. Concretely: `TIMEOUT / NOT COMPLETED` is rendered as a real, distinct status rather than a plausible-looking tick count, so a viewer can never mistake a failed run for a successful one; the negotiation feed presents each robot's decision as genuinely local rather than narrating a false central arbiter, which would misrepresent the very architecture the project is trying to prove out; and the Benchmark & Proof view's hard separation from live data (explicitly labeled, never blended) exists specifically so nobody — team member or judge — can be shown a live demo's numbers dressed up as the audited proof, or vice versa. A dashboard that quietly smoothed over any of these distinctions would be actively working against the project's own safety and honesty claims, not just failing to help them.

Role Chapter: Edge / ECE

1. Fundamentals

Sensor abstraction, from scratch. A robot's software shouldn't care whether "where am I" comes from a real GPS/LIDAR stack or a simulated ground-truth value with noise sprinkled on top — it should just call one interface (`read()`) and get a `Telemetry` reading back. This is what lets the exact same coordination code run unmodified whether the underlying sensor is simulated or (in an optional bonus path) real hardware.

Why "Edge-AI" doesn't require a neural network here. The problem statement's title uses "Edge-AI," but this project's category is Software with no physical hardware required — the documented position (also covered in the Problem Statement chapter) is that decentralized, local decision-making *on each robot* already qualifies as edge intelligence: each robot's agent reasons about its own situation using only its own sensor readings and messages it directly received, never a central brain. This role's actual deliverable is not a perception model — it's realistic fault injection.

Fault injection as the real engineering content of this role. Simulating noise, dropout, and full sensor failure isn't decoration bolted on to look more "sensor-like" — it's the specific mechanism that makes the deadlock-recovery and task-reassignment demo scenarios *actually testable* rather than hand-waved. Without a real way to make a robot's sensor genuinely go dark, "the system reassigns a failed robot's task" would be an untested claim.

Heartbeat monitoring, from scratch. A heartbeat monitor doesn't judge any single reading in isolation — it tracks a *streak*: how many consecutive readings in a row have been missing or unhealthy. Only once that streak crosses a threshold does it declare the robot genuinely offline. This is deliberately more forgiving than reacting to the very first bad reading, because a single dropped or noisy reading is normal, expected sensor behavior, not a failure.

2. This project's specific implementation

`edge/sensor_source.py:SimulationSensorSource` implements the abstract `SensorSource.read(robot_id, current_tick)`. It calls a `ground_truth_provider` callable to get the robot's real simulated position, then applies, in order: (a) `offline_after_tick` — if the current tick has reached this configured value, immediately returns an explicit `OFFLINE` telemetry reading, no dice roll involved; (b) `dropout_rate` — a random chance the reading is simply missing this tick (returns `None`, or an `OFFLINE` telemetry if the heartbeat monitor's own threshold is *also* crossed by this specific drop); (c) otherwise, a real reading, `DEGRADED` if `noise_std > 0` (Gaussian jitter applied to position, rounded back to an integer grid cell — the noisy value never breaks the "positions are always whole grid cells" contract) or `OK` if not.

`edge/fault_injection.py::SensorFaultConfig(noise_std, dropout_rate, offline_after_tick)` — validated strictly at construction (`ValueError` for a negative `noise_std`, an out-of- `[0,1]` `dropout_rate`, or a negative `offline_after_tick`), exactly mirroring `MessageFaultConfig` 's own fail-fast philosophy from the Communication role.

`edge/heartbeat.py::HeartbeatMonitor(miss_threshold=3)` — `record(robot_id, telemetry)` increments a per-robot miss counter whenever `telemetry` is `None` or its `sensorHealth == "OFFLINE"` ; any other reading (including `DEGRADED`) resets the counter to zero. `is_offline(robot_id)` is simply "has this robot's counter reached the threshold." One real, fixed bug is directly documented in the module's own current source comment: an earlier version treated `DEGRADED` the same as a dropped reading, so sustained noise alone (with zero actual dropout and no `offline_after_tick` set) would eventually and incorrectly force a robot offline — fixed by making only a genuinely missing or explicitly- `OFFLINE` reading count as a miss, since noise and dropout are meant to be independent, separately-configurable fault modes.

`edge/hardware/` — the optional, fully isolated hardware bonus path (a real sensor on a Raspberry Pi/ESP32, behind the identical `SensorSource` interface). Its absence or failure cannot affect the core demo, by construction: it is never imported by the default simulation path.

3. Alternatives considered and rejected

- **Truly random/uncorrelated noise with no ground-truth basis** — rejected: `SimulationSensorSource` deliberately derives every reading from the real simulated position first, then perturbs it, rather than generating disconnected synthetic values — this keeps a noisy reading meaningfully close to what the robot is "actually" doing, which matters for later logic (e.g. `RobotAgent`) that folds telemetry back into `RobotState.position` .
- **Reacting to the first bad reading instead of a streak-based threshold** — rejected: this would make a single unlucky dropped packet indistinguishable from a genuinely failed robot, causing false task-reassignment triggers on perfectly healthy hardware. A streak (default 3) absorbs normal transient noise.
- **Building actual physical hardware as the primary deliverable** — rejected outright per this role's own scope statement (`role/05_EDGE_ECE.md` §8.4: "explicitly not your job... don't attach hardware just to have something physical"); the fault-injection framework is the real, substantial engineering content of this role, and it was treated as such rather than as a placeholder while "the real work" happened elsewhere.

4. How this role communicates with every other role

To/From	Mechanism	What crosses
→ Coordination (Member 3)	<code>SensorSource.read()</code> , called inside <code>RobotAgent.tick()</code>	<code>Telemetry</code> — folded into <code>RobotState</code> every tick; <code>sensorHealth=OFFLINE</code> (or the heartbeat monitor's own <code>is_offline()</code>) drives <code>RobotState.status = OFFLINE</code>

To/From	Mechanism	What crosses
← Simulation Scenarios (Member 3)	constructor parameter	which <code>SensorFaultConfig</code> profile a scenario wants — e.g. <code>f_task_reassignment</code> 's <code>offline_robots</code> : <code>{"R1": 5}</code> , translated by <code>simulation/runner.py</code> into a real <code>SensorFaultConfig(offline_after_tick=5)</code> for that robot's own sensor instance
→ Backend (Member 6), indirectly via Coordination	<code>RobotState.status=OFFLINE</code> propagating through telemetry push	triggers <code>TaskAllocationService.handleRobotOffline()</code> on the Java side (see the Backend role chapter for the important nuance about what this does and doesn't reach)
→ Dashboard (Member 4), indirectly	<code>RobotState.status</code>	<code>OFFLINE</code> is rendered as a visually unmistakable status — a documented, explicit requirement in the shared contract itself

5. Evolution from the original role MD

`role/05_EDGE_ECE.md`'s specified interfaces (`SensorSource` , `SimulationSensorSource` , `SensorFaultConfig` , `HeartbeatMonitor`) match the real, shipped code closely, including constructor parameter names and defaults (`miss_threshold=3`). The one genuine, traceable evolution beyond the original spec: **the noise/dropout conflation bug**, described above, was found and fixed after the original implementation shipped — the original spec's §8.3 describes the heartbeat monitor's job ("track consecutive missed/degraded readings") in a way that, read literally, is exactly what caused the bug (treating a degraded-but-present reading as a miss); the *fixed* behavior — only a genuinely absent or explicitly-offline reading counts — is a real, deliberate refinement beyond the original text, not merely an implementation detail filled in. Three new regression tests were added specifically to lock this fix in place and prevent it regressing silently.

`simulation/runner.py`'s actual integration of `SimulationSensorSource` also evolved beyond a literal reading of the original spec in one concrete way: the spec's interface takes one `ground_truth_provider` per instance; the real runner constructs **one** `SimulationSensorSource` **per robot** (not one shared instance) specifically so `offline_after_tick` — a single value per instance — can still support a *per-robot* offline schedule (only R1 goes offline in `f_task_reassignment` , never R2) — a correct adaptation of the documented per-robot-parameterized design, not a deviation from it.

6. How this role contributes to deadlock avoidance and coordination safety

This role doesn't detect or resolve conflicts directly, but its fault-injection framework is what makes two entire categories of the project's safety claims testable at all: (a) `d_deadlock`'s engineered stall depends on `MessageFaultConfig` (Communication's role) working alongside realistic sensor behavior so the scenario isn't trivially resolved before a genuine standoff occurs; (b) the *entire* task-reassignment resilience story (`f_task_reassignment` , Category 3 of the benchmark) exists only because this role's `offline_after_tick` mechanism can reliably, deterministically make a robot go dark at an exact, chosen tick — without that reliability, "does the system recover from a robot failure" would be an

untestable, unfalsifiable claim rather than a benchmarked one (0 collisions, real hand-off, verified across 10 seeds).

Role Chapter: Backend

1. Fundamentals

Spring Boot and REST controllers, from scratch. Spring Boot is a framework for building Java web applications with minimal manual wiring — you declare a class as a `@RestController`, annotate a method `@GetMapping("/api/robots")`, and the framework handles routing an incoming HTTP request to it, converting the return value to JSON, and sending the response. This is why the backend's actual endpoint implementations are short, focused methods rather than hand-rolled HTTP parsing.

Why a backend needs to be a client *and* a server in the same request-response cycle. This backend has two distinct external relationships: it is a **REST client** to Python's control server (it *calls out* to start/stop scenarios), and a **REST server** that Python *calls into* to push telemetry. Both directions exist in the same running process — `SimulationClient` (outbound) and the ingestion controllers (inbound) are separate concerns that happen to live in one Spring Boot application.

WebSocket broadcast, from scratch. Once telemetry arrives via a normal REST POST, the same request handler can, before returning, push that data out to every currently-connected WebSocket client — "ingest and broadcast in the same request" means a viewer's dashboard sees new data the instant it's received, not on the next poll cycle.

ORM / JPA persistence, from scratch. Object-Relational Mapping lets Java code work with plain objects (`RobotEntity`, `TaskEntity`) while a framework (Hibernate, via Spring Data JPA) translates that into actual SQL against whatever database is configured — this is what makes "swap SQLite for Postgres" a configuration change rather than a rewrite: the Java code never writes SQL by hand.

2. This project's specific implementation

Controllers (all read directly, `backend/src/main/java/.../controller/`): `RobotController` (`GET /api/robots`, `GET /api/robots/{id}`), `WarehouseController` (`GET / POST /api/warehouse`, the `GET` returning `503 WAREHOUSE_MAP_UNAVAILABLE` before any run has pushed a map), `MetricsController` (`POST / GET /api/metrics`), `EventIngestController` (`POST /api/events` — persists and broadcasts every event in the same request; `GET /api/events?type=&limit=` — a later, approved read-only addition, explicitly *not* scoped to a single run since `EventEntity` carries no run/scenario tag), `SimulationControlController` (`POST /api/simulation/control`, `GET /api/simulation/status`), and `TelemetryIngestController` (`POST /api/telemetry/batch`).

`TelemetryIngestController.ingest()`, traced directly: for every `RobotState` in the batch, it validates required fields, upserts the in-memory `RobotCacheService`, persists a `RobotEntity`, samples one `TelemetryEntity` row every 10th tick (not every tick — a deliberate, documented storage choice), and — if that robot's `status` is `"OFFLINE"` — calls

`TaskAllocationService.handleRobotOffline(robotId)`. After the whole batch, it calls `allocatePendingTasks()` and broadcasts the batch over `/ws/live` — all in one request.

TaskAllocationService — a genuinely simple greedy/nearest-idle-robot heuristic, by explicit design (`role/06_BACKEND.md` §8.3 forbids adding optimization like the Hungarian algorithm until the simple version works first). `handleRobotOffline()` looks up whether the offline robot currently holds an active (`ASSIGNED` / `IN_PROGRESS`) task in the backend's *own* database, and if so, reassigns it to the nearest idle robot (Manhattan distance to the pickup position) and emits a real `TASK_REASSIGNED` event. `currentAssignmentsForSimulation()` is the method that actually closes the loop into Python: it gathers every currently active `Task` / `TaskAssignment` pair and shapes it for `POST /control/start`'s `taskAssignments` field.

SimulationClient — a thin `RestClient` wrapper around Python's `/control/start`, `/control/stop`, `/control/status`, with connect/read timeouts and every failure wrapped in a `SimulationEngineException` so a Python-side problem (unreachable, or a clean rejection like "already running") surfaces to the controller layer as a clear error, never an uncaught exception or a crash.

LiveUpdateBroadcaster + **LiveUpdateWebSocketHandler** — the former formats `{"type", "tick", "payload"}` envelopes and hands them to the latter, which tracks connected sessions in a `CopyOnWriteArraySet` and sends to each; a new connection immediately receives a `ROBOT_STATE_BATCH` snapshot of current cache state on connect, so a dashboard opening mid-run isn't starting from nothing.

An important, precise nuance about the "centralized task assignment" claim, verified directly by reading `TelemetryIngestController` and `TaskAllocationService` side by side: `handleRobotOffline()` genuinely fires on every real `OFFLINE` telemetry ingestion — it is not dead code — but it operates entirely on the backend's *own* `Task` / `TaskAssignment` database rows. It has no live channel to redirect an *already-running* Python simulation's actual robot movement; that's a documented, real, open gap (see the Judge Q&A and Benchmark Journey chapters). In practice, during a live run, the visible task hand-off a viewer sees comes from a separate, Python-internal reassignment mechanism inside `simulation/runner.py`, not from this service's own decision — even though this service's decision is real and does update Java's own records and broadcast its own event.

3. Alternatives considered and rejected

- **The Hungarian algorithm or another optimized assignment method, from day one** — rejected as premature per this role's own explicit scope (§8.3): the simple greedy heuristic is a reasonable MVP, and optimization was deliberately deferred rather than never considered.
- **Authentication/authorization** — deliberately not built; explicitly optional per this role's spec, "do not build unless there's a concrete, stated reason to," and no such reason arose.
- **Persisting every raw tick permanently** — rejected: the backend samples one `TelemetryEntity` row every 10 ticks per robot rather than one row per tick per robot, per the explicit design note (`role/06_BACKEND.md` §8.6) that this would otherwise be an unbounded, unnecessary storage cost — the in-memory cache (not the periodic sample table) is what actually serves "current state" queries.

- **A hardcoded SQLite-only persistence layer** — rejected in favor of JPA/Hibernate specifically so the SQLite→Postgres path stays a configuration change (see the Architecture Decisions chapter).

4. How this role communicates with every other role

To/From	Mechanism	What crosses
→ Coordination (Member 3, Python)	REST client (<code>SimulationClient</code>)	<code>POST /control/start {scenarioId, mode, seed, speed, taskAssignments}</code> , <code>POST /control/stop</code> , <code>GET /control/status</code>
← Coordination (Member 3, Python)	REST server (ingestion controllers)	<code>RobotState[]</code> batches, <code>SimulationEvent[]</code> , one <code>PerformanceMetric</code> per run
→ Frontend (Member 4)	REST + WebSocket	every <code>GET /api/...</code> response, and every <code>/ws/live</code> frame — the frontend's <i>only</i> data source
← Frontend (Member 4)	REST	simulation start/stop requests, task-creation requests
(indirectly) Edge (Member 5)	via ingested <code>RobotState.status</code>	an <code>OFFLINE</code> status reaching this backend is what triggers <code>handleRobotOffline()</code> — the backend never talks to Edge's code directly, only sees its effect on the data Python pushes

5. Evolution from the original role MD

`role/06_BACKEND.md` 's specified REST surface (§8.1) matches the real, shipped controllers almost exactly, including endpoint paths and request/response shapes. The concrete, traceable evolutions beyond the original spec:

- `POST /control/start` 's payload grew a `taskAssignments` field, and `SimulationClient.start()` grew a second, backward-compatible overload accepting it — not present in the original spec's §9 interface sketch (`public void start(String scenarioId, String mode, long seed, String speed)` , no assignments parameter). This was a deliberate, later, explicitly-approved addition to close a real integration gap (Java's centralized task data reaching the decentralized Python simulation at all), kept backward-compatible: the original 4-argument `start()` still exists and is still used directly by the existing `SimulationClientTest` .
- `GET /api/warehouse` and its `POST` counterpart were added beyond the original spec's read-only sketch (§8.1 lists only `GET /api/warehouse → WarehouseMap`) — the `POST` side was a necessary, later addition once it became clear Python needed a way to actually publish the map it's using, reusing the same push-ingestion pattern telemetry/events/metrics already established rather than inventing a new one.
- `GET /api/events?type=&limit=` is a later, explicitly-approved read-only addition on top of the original spec's ingestion-only `POST /api/events` — added for the frontend's event-timeline catch-up feature, with the important, disclosed caveat (found only once this endpoint was

actually built and used) that `EventEntity` carries no run/scenario identifier, so it cannot be scoped to "just this run" the way the original spec's ingestion-only design never had to consider.

- **The original spec's Definition of Done (§17)** describes task reassignment as something that "works end-to-end" — the more precise, current understanding (see section 2's nuance above) is that it works end-to-end *within this backend's own bookkeeping and event stream*, but does not yet have a live channel into an already-running Python simulation — a real refinement of what "works end-to-end" actually means, found through direct integration testing rather than assumed from the original text.

6. How this role contributes to deadlock avoidance and coordination safety

This backend makes no movement or conflict decisions, and — critically, by design — **must never become an accidental single point of failure for those decisions**. Every safety number in this document's Benchmark Journey chapter was produced with this backend's own URL pointed at a deliberately unreachable port, and the system still produced 0 collisions — proof that the safety-critical layer genuinely doesn't route through here. What this role *does* contribute to safety is resilience of the surrounding system: `_push_to_backend`-style failures on the Python side are swallowed by design, and symmetrically, this backend's own ingestion/control code never lets a malformed batch or an unreachable Python engine crash the process or corrupt cached state for other requests (`GlobalExceptionHandler`, verified by controller-level try/catch wrapping in `SimulationClient`) — a backend that could be knocked over by one bad request would itself become a real point of fragility for the dashboard experience, even though it was never load-bearing for collision safety itself.